Sistema de gestión de servicios bajo demanda

(PaTodo)
Documento de Especificaciones, Requerimientos y Criterios de Aceptación de Software (DERCAS)

Equipo de Desarrollo de Sistemas — PaTodo

Departamento de Desarrollo de Sistemas

Proyecto	PaTodo: plataforma que conecta clientes con trabajadores cercanos para la contratación de servicios generales, del hogar y de emergencias.
Versión del documento	1.0
Fecha	Octubre de 2026
Estado	En producción (API REST, frontend web y reglas de seguridad desplegadas)
Elaborado por	Equipo de Desarrollo de Sistemas — PaTodo
Fuente de verdad	`spec/openapi.yaml` y `spec/schemas/` (Especificación OpenAPI 3 y JSON Schemas)

Índice

1. Definiciones, acrónimos y abreviaturas **4**

2. Descripción del sistema solicitado **7**

3. Definición de actores **9**

4. Procesos a sistematizar **10**
4.1. Autenticación y gestión de cuentas . . . 10
4.2. Configuración del catálogo de servicios . . . 11
4.3. Publicación de trabajos . . . 11
4.4. Búsqueda de trabajos cercanos y notificación . . . 12
4.5. Envío y gestión de ofertas . . . 13
4.6. Aceptación de oferta y asignación . . . 14
4.7. Cálculo de rutas y seguimiento en tiempo real . . . 15
4.8. Conversación y coordinación . . . 16
4.9. Llamada de voz entre las partes . . . 17
4.10. Finalización del trabajo y calificación . . . 18
4.11. Pago simulado del trabajo . . . 19
4.12. Notificaciones . . . 20
4.13. Administración del sistema . . . 21

5. Diagrama de flujo de los procesos **22**
5.1. Registro y autenticación . . . 22
5.2. Configuración del catálogo de servicios . . . 23
5.3. Publicación de un trabajo . . . 24
5.4. Notificación a los prestadores de servicios . . . 24
5.5. Envío de ofertas . . . 24
5.6. Aceptación de una oferta . . . 25
5.7. Ejecución de un trabajo . . . 25
5.8. Conversación y coordinación . . . 26
5.9. Llamada de voz entre las partes . . . 27
5.10. Finalización y calificación . . . 29
5.11. Pago simulado del trabajo . . . 30
5.12. Notificaciones . . . 31
5.13. Administración del sistema . . . 32
5.14. Flujo general del sistema . . . 33
5.15. Ciclo de vida del trabajo . . . 34

6. Casos de uso **35**
6.1. Diagrama de casos de uso del sistema . . . 35
6.2. Casos de uso del módulo de autenticación . . . 38
6.2.1. Caso de uso “UC-01: Registrarse / Iniciar sesión” . . . 38
6.2.2. Caso de uso “UC-02: Crear perfil de usuario” . . . 39
6.2.3. Caso de uso “UC-03: Refrescar token de identidad” . . . 41
6.3. Casos de uso del módulo de trabajos . . . 42
6.3.1. Caso de uso “UC-04: Publicar trabajo” . . . 42
6.3.2. Caso de uso “UC-05: Buscar trabajos cercanos” . . . 44
6.3.3. Caso de uso “UC-06: Cancelar trabajo” . . . 45
6.3.4. Caso de uso “UC-07: Completar trabajo” . . . 46

6.4. Casos de uso del módulo de ofertas 47
6.4.1. Caso de uso “UC-08: Enviar oferta” 47
6.4.2. Caso de uso “UC-09: Aceptar oferta” 48
6.5. Casos de uso del módulo de rutas 50
6.5.1. Caso de uso “UC-10: Calcular ruta (vista previa)” 50
6.5.2. Caso de uso “UC-11: Calcular ruta oficial” 50
6.6. Casos de uso del módulo de finalización 52
6.6.1. Caso de uso “UC-12: Crear reseña” 52
6.7. Casos de uso del módulo de seguimiento y chat 53
6.7.1. Caso de uso “UC-13: Seguimiento en tiempo real y chat” 53
6.8. Casos de uso del módulo de notificaciones 54
6.8.1. Caso de uso “UC-14: Recibir y consultar notificaciones” 54
6.9. Casos de uso del módulo de comunicación por voz 56
6.9.1. Caso de uso “UC-15: Iniciar llamada de voz” 56
6.9.2. Caso de uso “UC-16: Finalizar llamada de voz” 58
6.10. Casos de uso del módulo de administración 59
6.10.1. Caso de uso “UC-17: Consultar panel de métricas” 59
6.10.2. Caso de uso “UC-18/UC-19: Gestionar usuarios y roles” 59
6.10.3. Caso de uso “UC-20: Promover / revocar administrador” 60
6.10.4. Caso de uso “UC-21: Suspender / activar cuenta” 61
6.10.5. Caso de uso “UC-22: Verificar profesional” 62
6.10.6. Caso de uso “UC-23: Mantener catálogo de categorías y habilidades” . 63
6.10.7. Caso de uso “UC-24: Consultar registro de auditoría” 64

7. Diagrama de secuencia **66**
7.1. Secuencia de publicación de un trabajo 66
7.2. Secuencia de aceptación de oferta 66
7.3. Secuencia de finalización y calificación 67
7.4. Secuencia de la búsqueda de trabajos cercanos 67

8. Diagramas de estado **68**
8.1. Estados del trabajo 68
8.2. Estados de la oferta 68
8.3. Estados de la llamada de voz 69

9. Diagrama ER **69**

10. Requerimientos no funcionales **70**
10.1. Arquitectura de tres capas 70
10.2. Autenticación y autorización 71
10.3. Seguridad de datos 72
10.4. Almacenamiento 73
10.5. Integración 73

11. Planificación de ciclos de desarrollo **74**
11.1. Estado de avance 75

12. Firmas de aceptación **76**

1. Definiciones, acrónimos y abreviaturas

Término	**Concepto**
API	Interfaz de programación de aplicaciones. Pieza de código que permite que dos aplicaciones se comuniquen entre sí y compartan información y funcionalidades; es el intermediario que permite que un sistema pida datos o ejecute acciones específicas. En PaTodo la implementa el servicio REST en Express, desplegado en Render.
Backend	Arquitectura interna de una aplicación, no visible para el usuario. Se encarga de la lógica de negocio, de recibir peticiones y devolver datos procesados. En PaTodo lo componen los servicios gestionados de Firebase y la API REST transaccional.
Browser	Navegador web o explorador de Internet.
Casos de uso	Descripción de una acción o actividad. El diagrama de casos de uso describe las actividades que debe realizar alguien o algo para llevar a cabo un proceso dentro del sistema.
Client (SDK)	Bibliotecas oficiales de Firebase para el cliente (Web y móvil) que permiten autenticación, lectura en tiempo real y escritura directa sobre documentos propios del usuario, siempre validadas por las reglas de seguridad.
CORS	*Cross-Origin Resource Sharing*. Mecanismo que permite que un navegador acepte peticiones desde orígenes distintos al que sirve la aplicación; en PaTodo se restringe mediante una lista blanca en la variable `CORS_ORIGINS`.
CRUD	Siglas de *Crear, Leer, Actualizar y Eliminar*; las cuatro operaciones básicas sobre los datos de una aplicación.

Término	**Concepto**
Custom Claim	Declaración personalizada que viaja dentro del ID token de Firebase Authentication. PaTodo la usa para transportar el rol (`role`) hasta las reglas de seguridad de Firestore.
DERCAS	*Documento de Especificaciones, Requerimientos y Criterios de Aceptación de Software*. Documento que describe el sistema solicitado, sus actores, procesos, casos de uso, requisitos y criterios de aceptación.
Docker	Herramienta que automatiza el despliegue de aplicaciones, permitiendo crear, probar e implementar de forma reproducible.
FCM	*Firebase Cloud Messaging*. Servicio de notificaciones push que entrega avisos a los dispositivos de los usuarios; en PaTodo lo usa la API REST mediante el Admin SDK.
Firestore	Base de datos de documentos de Google Firebase, con escalado automático, consultas en tiempo real y reglas de seguridad declarativas. Es el almacén de datos de PaTodo.
Frontend	Parte de una aplicación que interactúa con los usuarios; es el lado del cliente, es decir, lo que se ve en pantalla. En PaTodo existe una versión web (React) y una móvil (Flutter).
GeoPoint / Geohash	Tipo de dato de Google para representar coordenadas geográficas y, respectivamente, su codificación de cadena corta. PaTodo almacena el `geohash` dentro del `location` de los usuarios y los trabajos para habilitar búsquedas por proximidad.
Geofire (geofire-common)	Librería que calcula el `geohash` de un `GeoPoint` y los límites del área (`geohashQueryBounds`) que la cubre, base de la búsqueda de trabajos cercanos.

Término	**Concepto**
Haversine	Fórmula trigonométrica para calcular la distancia en línea recta entre dos coordenadas geográficas. PaTodo la usa para descartar los trabajos que caen fuera del radio solicitado.
HTTPS	Protocolo de transferencia de hipertexto seguro; destino cifrado de las comunicaciones.
ID Token	Token de identidad que emite Firebase Authentication tras iniciar sesión. PaTodo lo envía el cliente en la cabecera `Authorization: Bearer` y la API lo verifica con el Admin SDK.
OpenAPI 3	Especificación estándar (YAML) que describe de forma formal las operaciones HTTP de una API: rutas, parámetros, esquemas de entrada y respuestas. En PaTodo vive en `spec/openapi.yaml` y es la fuente única de verdad de los contratos.
OSRM	*Open Source Routing Machine*. Motor de cálculo de rutas por carretera. PaTodo lo consulta la API REST para obtener la geometría, la distancia y la duración del trayecto.
Patrón MVC	*Modelo-Vista-Controlador*. Separación del software en modelo (datos), vista (interfaz) y controlador (lógica).
PWA / SPA	*Progressive Web Application* / *Single Page Application*: aplicación web que ejecuta su contenido en una sola página, sin recargas completas.
REST	Interfaz para conectar sistemas sobre HTTP; obtiene y genera datos devolviéndolos en formatos definidos, como JSON.
SDK	*Software Development Kit* (kit de desarrollo de software).
Spec-Driven Development (SDD)	Metodología adoptada en PaTodo: primero se define el contrato en la especificación (`spec/`) y después se implementa en la API y en los frontends.

Término	Concepto
SSO	*Single Sign-On* (inicio de sesión único): procedimiento que habilita a un usuario para acceder a varios sistemas con una sola autenticación.
TURN / STUN	Servidores de apoyo del trayecto de una conexión WebRTC: STUN descubre la dirección pública del dispositivo y TURN la retransmite cuando la conexión directa no es posible (por ejemplo, detrás de CGNAT).
WebRTC	Tecnología de navegador que permite comunicación de voz y video en tiempo real entre dos puntos (peer-to-peer), sin pasar por un servicio de transmisión central.
XLSX / PDF	Formatos de hoja de cálculo y de documento portátil, respectivamente.

2. Descripción del sistema solicitado

En la actualidad, la contratación de servicios generales, del hogar y de emergencia (mecánica, plomería, jardinería, pintura, instalación eléctrica, etc.) depende de contactos informales, llamadas telefónicas o anuncios aislados, lo que dificulta encontrar un profesional disponible cerca de la ubicación del cliente, comparar precios y evaluar la calidad del trabajo realizado.

Se solicita el desarrollo e implementación de una plataforma de servicios bajo demanda, denominada **PaTodo**, que conecte clientes con trabajadores independientes cercanos, permitiendo que el cliente publique un trabajo, que los trabajadores cercanos lo descubran y envíen ofertas, y que el cliente elija la mejor opción. El sistema debe permitir:

- El registro y la autenticación de clientes y trabajadores, con una plataforma web y una aplicación móvil.
- La publicación de trabajos, especificando descripción, categoría, habilidades requeridas, ubicación (recogida y destino opcional), precio propuesto y fecha programada.

- La búsqueda de trabajos cercanos al trabajador por proximidad geográfica, con filtros de categoría y radio de búsqueda.
- El envío de ofertas por parte de los trabajadores, con precio, tiempo estimado y mensaje, sobre trabajos ajenos y en estado pendiente.
- La aceptación de una oferta por parte del cliente, lo que asigna al trabajador seleccionado y abre automáticamente una conversación privada.
- La cancelación del trabajo por parte del cliente mientras no haya sido completado.
- El seguimiento de la ubicación del trabajador en tiempo real y el cálculo de la ruta entre los puntos del trayecto.
- La finalización del trabajo y la calificación mutua entre cliente y trabajador, con puntuación y comentario.
- Las notificaciones en tiempo real de los eventos relevantes (nuevas ofertas, aceptación, finalización, calificación).
- La comunicación de voz entre las partes del trabajo mediante conexión WebRTC punto a punto, con señalización persistida en la base de datos.
- Un módulo de administración para la gestión de usuarios, verificación de profesionales, suspensión y activación de cuentas, promoción de administradores y mantenimiento del catálogo de categorías y habilidades.

El sistema se construirá sobre una arquitectura de servicios gestionados en la nube: autenticación, base de datos de documentos, notificaciones push y almacenamiento en Firebase, más una API REST transaccional propia en Node.js/Express responsable de las operaciones atómicas. Las interfaces de usuario se comunican con estos servicios mediante los SDK oficiales y la API REST.

El desarrollo se apoya, además, en un modelo de desarrollo orientado a especificaciones (*Spec-Driven Development*), de modo que los contratos de datos y de API se documentan formalmente antes de implementarse y constituyen la fuente única de verdad del proyecto.

3. Definición de actores

Nombre del actor	**Descripción**
Cliente (`client`)	Usuario que contrata servicios. Publica trabajos, revisa las ofertas recibidas, acepta la oferta que mejor le convenga, cancela trabajos no finalizados, conversa con el trabajador asignado, sigue su ubicación, finaliza el trabajo y califica al profesional.
Trabajador (`worker`)	Usuario que ofrece servicios. Consulta los trabajos cercanos a su ubicación mediante búsqueda geográfica, calcula la ruta hacia el trabajo, envía ofertas, gestiona sus propias ofertas (editar o retirar), reporta su ubicación durante el trayecto, finaliza el trabajo y califica al cliente.
Ambos (`both`)	Usuario que puede actuar indistintamente como cliente y como trabajador. La aplicación le habilita ambos conjuntos de acciones según el contexto de la pantalla.
Administrador (`admin`)	Superusuario del sistema. Accede al panel de administración para consultar métricas de operación, listar y filtrar usuarios y trabajos, cambiar roles, promover o revocar administradores, suspender o activar cuentas, verificar profesionales y mantener el catálogo de categorías y habilidades.
API REST (actor interno)	Componente de servidor que actúa como actor del sistema. Verifica el ID token de cada petición, ejecuta las transacciones que exigen atomicidad (aceptar oferta, cancelar, completar, calcular ruta, crear reseña, sesión de voz) y genera las notificaciones correspondientes.
Reglas de seguridad de Firestore (actor interno)	Componente que autoriza las escrituras directas de los clientes a partir del rol declarado en el token y de la propiedad del recurso.

4. Procesos a sistematizar

A continuación se describen los procesos de negocio que el sistema debe automatizar. Cada proceso indica su descripción, los actores involucrados, sus entradas y salidas, las especificaciones de funcionamiento y las restricciones adicionales que condicionan su implementación.

4.1. Autenticación y gestión de cuentas

Nombre	Autenticación y gestión de cuentas
Descripción	Registro e inicio de sesión de clientes y trabajadores mediante correo electrónico y contraseña, así como mediante Google Sign-In. Una vez autenticado, el usuario completa su perfil (nombre, teléfono, foto de perfil) y selecciona el rol con el que operar (`client`, `worker` o `both`), lo que genera el documento `users/{uid}` y asigna el Custom Claim `role`.
Actores	Cliente, Trabajador, API REST
Entradas	Correo electrónico, contraseña, o credenciales de Google; nombre, teléfono y rol seleccionado.
Salidas	Sesión iniciada con ID token; documento de usuario creado; Custom Claim `role` asignado.
Especificaciones de funcionamiento	El registro y el inicio de sesión se resuelven directamente contra Firebase Authentication desde el cliente. La creación del perfil se realiza mediante `POST /createUser`, que valida que el `uid` y el correo coincidan con los del token, crea el documento de usuario y asigna el rol como *Custom Claim*. Al finalizar, el cliente debe refrescar su ID token (`getIdToken(true)`) para que el rol viaje en las peticiones. Los usuarios previos a la asignación del rol se actualizan con la utilidad de sincronización de claims.

Restricciones adicionales	Un usuario no puede publicar trabajos con el rol `worker` ni enviar ofertas con el rol `client`. No puede crearse un segundo perfil para el mismo `uid`. El teléfono es obligatorio para completar el perfil.

4.2. Configuración del catálogo de servicios

Nombre	Configuración del catálogo de servicios
Descripción	Administración de las colecciones maestras `categories` y `skills`, que sustentan la clasificación de los trabajos publicados. Cada habilidad se asocia a una o más categorías mediante `categoryIds`.
Actores	Administrador
Entradas	Datos de la categoría (código, nombre, descripción, icono, color, imagen, orden, estado activo) y de la habilidad (código, nombre, categorías asociadas, estado activo).
Salidas	Catálogos actualizados, disponibles para los formularios de publicación de trabajos y para los filtros de búsqueda.
Especificaciones de funcionamiento	La lectura de ambos catálogos es pública, incluso sin autenticar. La creación, actualización y eliminación está reservada al rol `admin`, tanto en las reglas de seguridad como en el panel web. El despliegue inicial se realiza con el catálogo semilla (8 categorías y 12 habilidades), una operación no destructiva que realiza *upsert* con identificadores deterministas iguales al `slug` del elemento.
Restricciones adicionales	Un cliente o trabajador nunca puede modificar el catálogo. La operación de siembra de pruebas que borra la base de datos está prohibida en producción.

4.3. Publicación de trabajos

Nombre	Publicación de trabajos
Descripción	El cliente publica una solicitud de servicio indicando en qué necesita ayuda, en qué categoría, qué habilidades se requieren, dónde se prestará el servicio (lugar de recogida), si hay un punto de entrega, cuánto está dispuesto a pagar y, opcionalmente, la fecha en que se requiere.
Actores	Cliente, Trabajador (destinatario), API REST (notificación)
Entradas	Descripción del trabajo, `categoryId`, `skillIds`, ubicación con coordenadas y `geohash`, precio propuesto, destino opcional, fecha programada opcional.
Salidas	Documento `jobs/{id}` en estado `pending`, visible en la búsqueda de trabajos cercanos.
Especificaciones de funcionamiento	La publicación se realiza con escritura directa del cliente sobre Firestore. La regla de seguridad exige rol `client` o `both`, que la cuenta no esté suspendida, que `clientId` coincida con el `uid` del token y que el trabajo nazca en estado `pending`. El cliente puede editar únicamente `details`, `pricing`, `location` y `updatedAt`, y puede eliminar el trabajo únicamente mientras siga en estado `pending`; cualquier otro cambio de estado se realiza exclusivamente a través de la API.
Restricciones adicionales	El cliente no puede asignar `workerId`, `acceptedOfferId`, `status` ni `route` por escritura directa; hacerlo le permitiría auto-completar el trabajo y fabricar reseñas.

4.4. Búsqueda de trabajos cercanos y notificación

Nombre	Búsqueda de trabajos cercanos y notificación

Descripción	El trabajador consulta los trabajos pendientes próximos a su ubicación, filtrados por radio de búsqueda y, opcionalmente, por categoría.
Actores	Trabajador, API REST
Entradas	Latitud, longitud, radio en kilómetros (por defecto 10, máximo 50), categoría opcional y límite de resultados (por defecto 20, máximo 50).
Salidas	Listado de trabajos con la distancia en línea recta expresada en kilómetros, ordenada de menor a mayor.
Especificaciones de funcionamiento	La búsqueda se realiza mediante `GET /jobs/nearby`. El servicio calcula los límites geográficos del radio solicitado, consulta los trabajos en estado `pending` cuyo `location.geohash` cae dentro de esos límites mediante el índice compuesto de la base de datos y descarta los que superen el radio tras aplicar la fórmula de Haversine. La distancia mostrada es en línea recta y **no** representa la ruta por carretera.
Restricciones adicionales	Solo los roles `worker` y `both` pueden consultar; un cliente recibe `403`. Un trabajo sin `location.geohash` publicado no aparece en los resultados. Los parámetros con formato inválido o fuera de rango se rechazan con `400`.

4.5. Envío y gestión de ofertas

Nombre	Envío y gestión de ofertas
Descripción	El trabajador envía una oferta económica y operativa para un trabajo cercano: precio propuesto, tiempo estimado de ejecución y mensaje para el cliente.
Actores	Trabajador, Cliente

Entradas	`jobId`, precio, tiempo estimado, moneda, mensaje.
Salidas	Documento `offers/{id}` en estado `pending`, visible para el cliente dueño del trabajo.
Especificaciones de funcionamiento	El envío se realiza con escritura directa del cliente. La regla exige rol `worker` o `both`, que `workerId` coincida con el `uid` del token, que la oferta nazca en estado `pending` y que el trabajo referenciado esté en estado `pending` y **no pertenezca al propio trabajador**. El trabajador puede editar precio, mensaje, tiempo estimado, moneda y estado de su propia oferta; el único cambio de estado permitido por escritura directa es de `pending` a `withdrawn`.
Restricciones adicionales	Un trabajador no puede ofertar por su propio trabajo. No puede modificar el `jobId` ni el `workerId`, ni auto-aceptar su oferta. Las ofertas no se borran: se retiran mediante el estado `withdrawn`.

4.6. Aceptación de oferta y asignación

Nombre	Aceptación de oferta y asignación
Descripción	El cliente selecciona, entre las ofertas recibidas, la que mejor le conviene según precio, tiempo estimado, perfil y calificación del trabajador.
Actores	Cliente, API REST
Entradas	`jobId`, `offerId`.
Salidas	Trabajo en estado `accepted` con el trabajador asignado; resto de ofertas en `rejected`; conversación privada creada; notificación push a las partes.

Especificaciones de funcionamiento	La operación se ejecuta en una única transacción en el servidor (`POST /acceptOffer`) que valida que el llamante sea el cliente dueño del trabajo y que el trabajo esté en estado `pending`. Dentro de la transacción se marca la oferta seleccionada como `accepted`, el resto de ofertas pendientes como `rejected`, el trabajo como `accepted` registrando el `workerId` y el `acceptedOfferId`, y se crea la conversación asociada al trabajo en estado `active`.
Restricciones adicionales	Solo el cliente dueño del trabajo puede aceptar una oferta. Un trabajo con ofertas ya aceptadas no puede aceptar otra. La operación es idempotente frente a intentos concurrentes gracias al cierre transaccional.

4.7. Cálculo de rutas y seguimiento en tiempo real

Nombre	Cálculo de rutas y seguimiento en tiempo real
Descripción	Cuando un trabajador consulta un trabajo pendiente, la plataforma le muestra una vista previa de la distancia y del trayecto para decidir si le interesa. Una vez asignado, la ruta definitiva se calcula y se persiste para que el cliente pueda ver el trayecto. Durante la ejecución, el trabajador reporta su ubicación y el cliente la sigue en tiempo real.
Actores	Trabajador, Cliente, API REST
Entradas	`jobId`; para el seguimiento, las coordenadas del dispositivo del trabajador.
Salidas	Vista previa (geometría, distancia, duración) sin persistir; ruta oficial almacenada en el campo `route` del trabajo; documento de ubicación actual del seguimiento.

Especificaciones de funcionamiento	El servicio `POST /computeRoute` opera en dos modos según el rol del llamante y el estado del trabajo. En modo *vista previa*, un trabajador no asignado consulta un trabajo pendiente y obtiene la ruta desde su ubicación hasta el lugar de recogida, sin escribir ningún dato. En modo *oficial*, el cliente dueño o el trabajador asignado de un trabajo aceptado obtienen la ruta desde la ubicación del trabajador hasta el lugar de recogida y, si existe, hasta el punto de entrega; el resultado se guarda en el trabajo. El trayecto se divide en tramos cuando existe destino. La posición en vivo del trabajador se escribe en el documento de seguimiento del trabajo, legible únicamente por el cliente dueño y el trabajador asignado, y se elimina al completar o cancelar el trabajo.
Restricciones adicionales	La ruta oficial solo se calcula para trabajos aceptados. El trabajador debe tener una ubicación válida con coordenadas geográficas actualizadas. Si el servicio de cálculo de rutas no responde, la operación falla con un error de servicio no disponible. No se publica la ubicación precisa del usuario en su perfil general.

4.8. Conversación y coordinación

Nombre	Conversación y coordinación
Descripción	Al aceptar una oferta se abre un chat privado entre el cliente y el trabajador asignado para coordinar los detalles del servicio.
Actores	Cliente, Trabajador
Entradas	Mensajes de texto con indicación de tipo y, opcionalmente, respuesta a un mensaje previo.
Salidas	Mensajes almacenados en la conversación; indicador de mensajes no leídos y resumen del último mensaje.

Especificaciones de funcionamiento	Existe exactamente una conversación por trabajo, creada por la API al aceptar la oferta. Los mensajes se escriben directamente por los participantes mediante el SDK del cliente, y la regla de seguridad exige que el remitente sea participante de la conversación. El resumen del último mensaje y la marca de última lectura solo pueden actualizarse por los participantes; los mensajes son inmutables una vez escritos.
Restricciones adicionales	Solo los participantes pueden leer y escribir en la conversación y sus mensajes; no pueden consultarla ni terceras partes. El tamaño de cada documento está limitado para evitar abuso. No es posible editar ni eliminar mensajes.

4.9. Llamada de voz entre las partes

Nombre	Llamada de voz entre las partes
Descripción	El cliente y el trabajador asignado pueden iniciar una llamada de voz directa mientras el trabajo se encuentra en curso. La señalización se persiste en la base de datos y el audio viaja directamente entre los dispositivos mediante WebRTC.
Actores	Cliente, Trabajador, API REST
Entradas	`jobId` para iniciar; `callId`, estado final y duración para finalizar. Mensajes de señalización (oferta, respuesta, candidatas de conexión, colgado).
Salidas	Sesión de llamada creada con identificador, contraparte, servidores de conexión y ruta de señalización; historial de llamadas con estado, duración y tipo de relevo de medios.

Especificaciones de funcionamiento	La solicitud de sesión se procesa mediante `POST /createVoiceSession`, que valida que el trabajo exista, se encuentre aceptado o en curso, que el llamante sea el cliente o el trabajador del trabajo y que ninguno esté suspendido. La API registra la llamada en estado *llamando* y toma un cerrojo transaccional por trabajo, devolviendo al cliente los servidores de conexión con credenciales de tiempo limitado. La señalización (oferta, respuesta y candidatas de conexión) la escriben los dos participantes en los documentos de la llamada. Al finalizar, `POST /endVoiceCall` valida que el llamante sea parte de la llamada y que la transición de estado sea válida, escribe el desenlace y elimina la señalización y el cerrojo.
Restricciones adicionales	Solo puede existir una llamada activa por trabajo: un segundo intento concurrente se rechaza con conflicto. No se permiten llamadas a números telefónicos; el audio se establece directamente entre los participantes y **nunca** se almacena. El audio viaja cifrado. Las llamadas entrantes en segundo plano solo están soportadas en Android; en iOS quedan fuera de la primera versión. Una llamada que no se atiende dentro del tiempo de espera configurado se cierra automáticamente como perdida y libera el cerrojo.

4.10. Finalización del trabajo y calificación

Nombre	Finalización del trabajo y calificación
Descripción	Cuando el servicio se presta, el trabajo se marca como completado por cualquiera de las dos partes y ambos pueden dejar una calificación sobre la contraparte.
Actores	Cliente, Trabajador, API REST

Entradas	`jobId`; para la reseña, puntuación de 1 a 5 y comentario opcional.
Salidas	Trabajo en estado `completed`; reseña registrada; calificación promedio y número de reseñas actualizados en el perfil del evaluado; notificación a la contraparte.
Especificaciones de funcionamiento	La finalización se ejecuta mediante `POST /completeJob`, en una transacción que valida que el llamante sea el cliente o el trabajador asignado y que el trabajo esté aceptado. La calificación se registra mediante `POST /createReview`, que valida que el autor sea participante del trabajo, que el trabajo esté completado y que el autor no haya calificado previamente ese mismo trabajo; escribe la reseña, recalcula la calificación promedio y el conteo del perfil del evaluado y genera la notificación correspondiente.
Restricciones adicionales	Solo los participantes de un trabajo completado pueden calificar, y una sola vez por trabajo y por persona. Las reseñas son inmutables: no pueden modificarse ni eliminarse. La calificación influye directamente en la reputación del profesional.

4.11. Pago simulado del trabajo

Nombre	Pago simulado del trabajo
Descripción	Cuando el trabajo está aceptado, en ejecución o completado, el cliente registra el pago del servicio mediante un módulo demostrativo que no mueve dinero real.
Actores	Cliente, API REST
Entradas	`jobId` y método de pago (`demo_card`, `demo_cash` o `demo_bank`).
Salidas	Documento de pago en estado `pagado` con la marca `isDemo`, asociado al trabajo; comisión de plataforma y pago neto calculados; notificación al trabajador.

Especificaciones de funcionamiento	El pago se registra mediante `POST /createPayment`, que valida que el llamante sea el cliente dueño del trabajo, que exista un trabajador asignado y que el estado del trabajo permita el pago. En una transacción calcula el monto a partir del precio de la oferta aceptada o del precio propuesto, aplica la comisión de plataforma del 10 % para obtener el pago neto, crea el pago con estado `pagado` y la marca `isDemo`, lo asocia al trabajo y notifica al trabajador.
Restricciones adicionales	El módulo es demostrativo: no procesa tarjetas reales ni transfiere dinero. Solo el cliente dueño del trabajo puede pagarlo, y un trabajo ya pagado rechaza un segundo intento.

4.12. Notificaciones

Nombre	Notificaciones
Descripción	El sistema avisa a los usuarios sobre los eventos relevantes de sus trabajos y ofertas.
Actores	API REST (emisor), Cliente, Trabajador, Administrador (destinatario)
Entradas	Eventos del dominio: oferta recibida, oferta aceptada o rechazada, trabajo cancelado, trabajo completado, reseña recibida, llamadas.
Salidas	Documento en la colección `notifications` del destinatario y notificación push en su dispositivo.
Especificaciones de funcionamiento	Las notificaciones las crea exclusivamente la API REST después de confirmar la transacción que origina el evento. Cada usuario puede leer únicamente sus propias notificaciones y marcar como leídas las que aún no lo están. La entrega al dispositivo se realiza mediante el servicio de mensajería push de Firebase, usando los tokens registrados en el perfil del usuario.

Restricciones adicionales	Las notificaciones no pueden ser creadas, modificadas ni eliminadas por el cliente. El destinatario solo puede actualizar la marca de lectura.

4.13. Administración del sistema

Nombre	Administración del sistema
Descripción	El administrador gestiona las cuentas de la plataforma, supervisa la operación y mantiene el catálogo de servicios.
Actores	Administrador
Entradas	Identificador del usuario a gestionar; rol a asignar; estado (suspender o activar); verificación del profesional; filtros de consulta (rol, estado, texto de búsqueda, página, límite).
Salidas	Panel con métricas de operación, listados paginados de usuarios y trabajos, perfil actualizado, registro de auditoría de acciones administrativas.
Especificaciones de funcionamiento	Todas las operaciones administrativas se ejecutan por la API, que verifica el rol `admin` del token antes de actuar. El módulo permite consultar métricas (usuarios por rol, trabajos por estado, ofertas, calificación promedio y actividad de la última semana), listar y filtrar usuarios y trabajos, cambiar el rol de un usuario entre las tres categorías operativas, promover y revocar administradores, suspender y activar cuentas y verificar o dejar de verificar profesionales. Todas las acciones quedan registradas en la colección de auditoría con el usuario, la acción, la entidad afectada y una descripción.

Restricciones adicionales	El rol de administrador no se puede asignar mediante el cambio de rol convencional; requiere la operación específica de promoción. Un administrador no puede revocarse a sí mismo, de modo que siempre exista al menos un administrador operativo. No se puede suspender a un administrador. La cuenta suspendida conserva la lectura pero pierde toda capacidad de escritura (publicar, ofertar, editar perfil y vehículos). El registro de auditoría solo puede ser leído por administradores.

5. Diagrama de flujo de los procesos

Los diagramas de flujo describen la secuencia de decisiones y actividades de los procesos principales del sistema.

5.1. Registro y autenticación

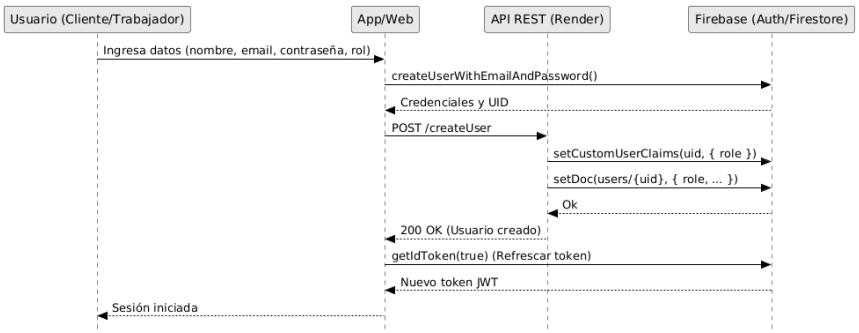


Figura 1

Diagrama de flujo del proceso de registro y autenticación de un usuario.

5.2. Configuración del catálogo de servicios

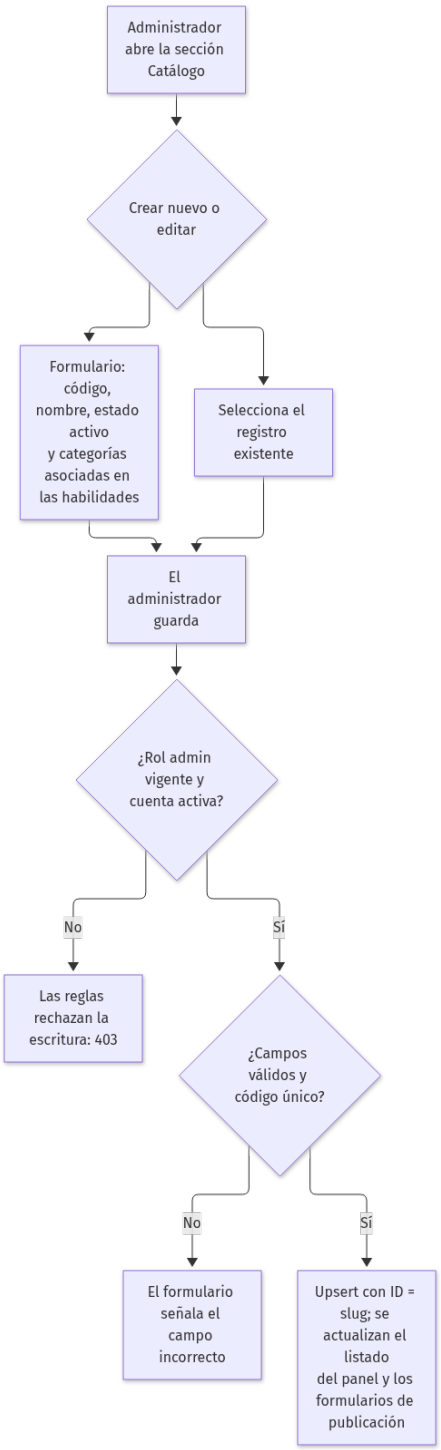


Figura 2

Diagrama de flujo del proceso de mantenimiento del catálogo de categorías y habilidades. La lectura es pública y la escritura corresponde al administrador.

5.3. Publicación de un trabajo

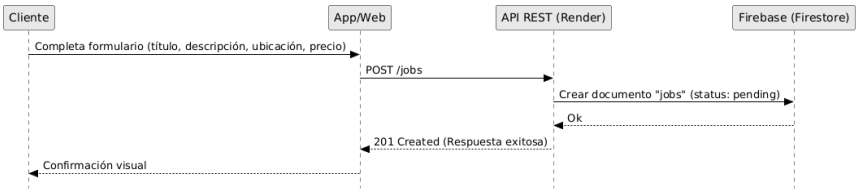


Figura 3

Diagrama de flujo del proceso de publicación de un trabajo por parte del cliente.

5.4. Notificación a los prestadores de servicios

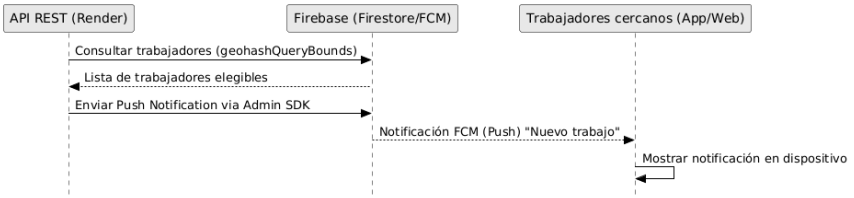


Figura 4

Diagrama de flujo del descubrimiento de un trabajo publicado por parte de los trabajadores cercanos.

5.5. Envío de ofertas

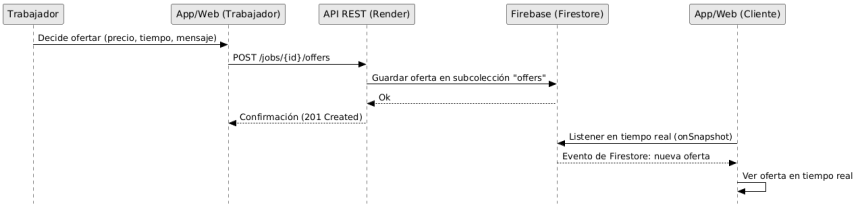


Figura 5

Diagrama de flujo del proceso de envío de una oferta por parte del trabajador.

5.6. Aceptación de una oferta

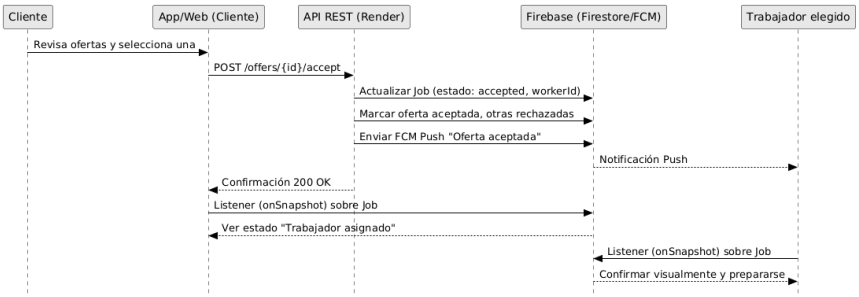


Figura 6

Diagrama de flujo del proceso de aceptación de oferta y asignación del trabajador.

5.7. Ejecución de un trabajo

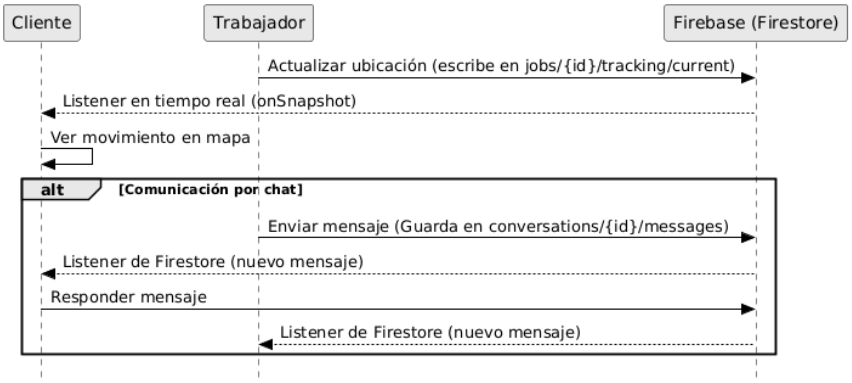


Figura 7

Diagrama de flujo del proceso de ejecución del trabajo, seguimiento y coordinación.

5.8. Conversación y coordinación

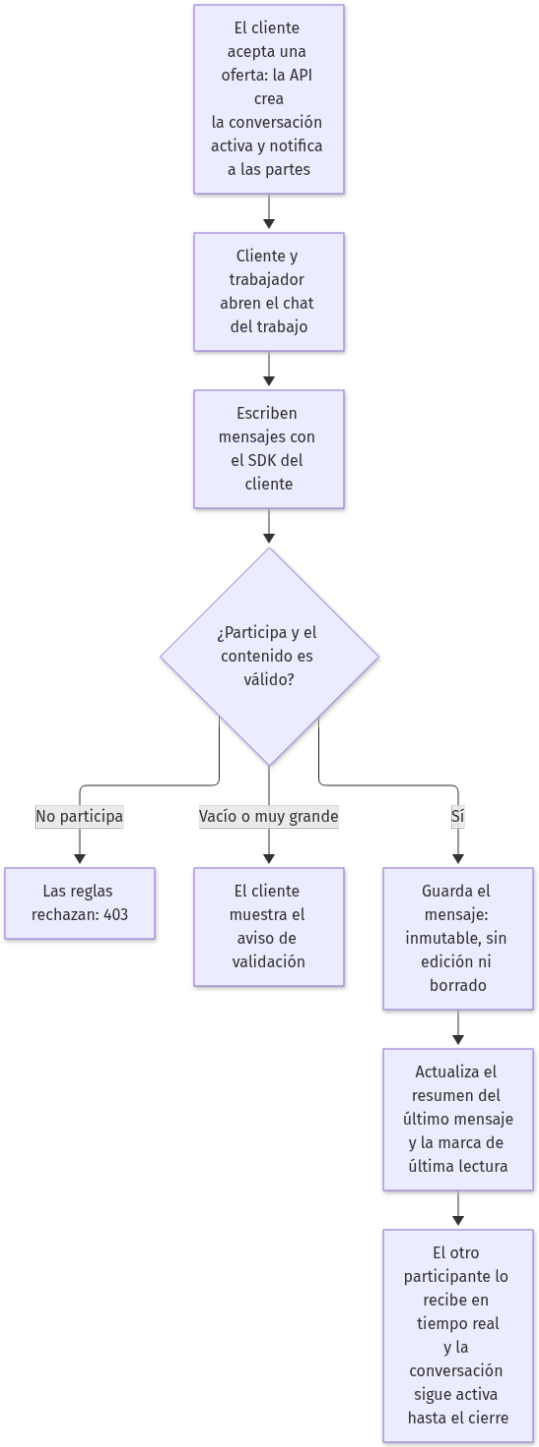


Figura 8

Diagrama de flujo del proceso de mensajería entre cliente y trabajador dentro de la conversación del trabajo.

5.9. Llamada de voz entre las partes

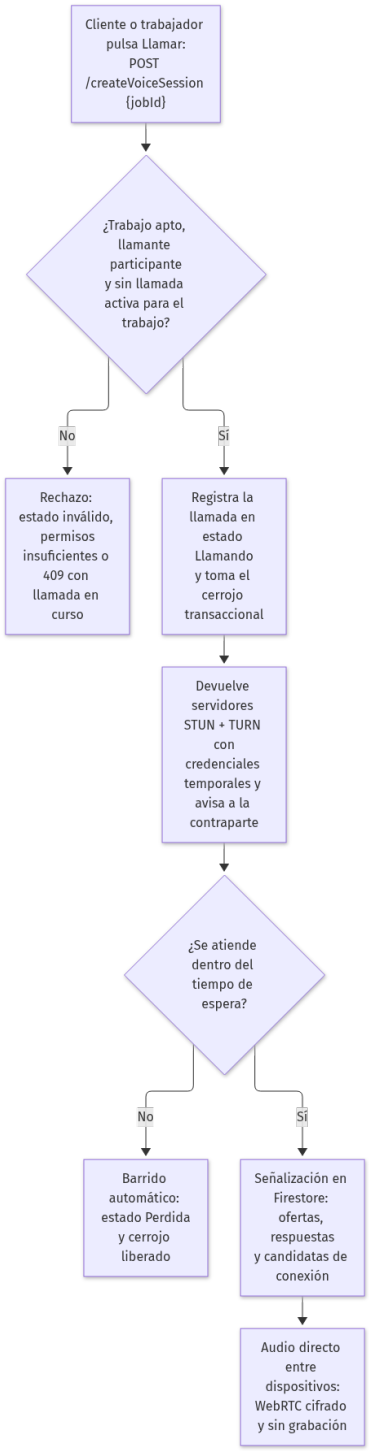


Figura 9

Diagrama de flujo de la solicitud y el establecimiento de una llamada de voz.

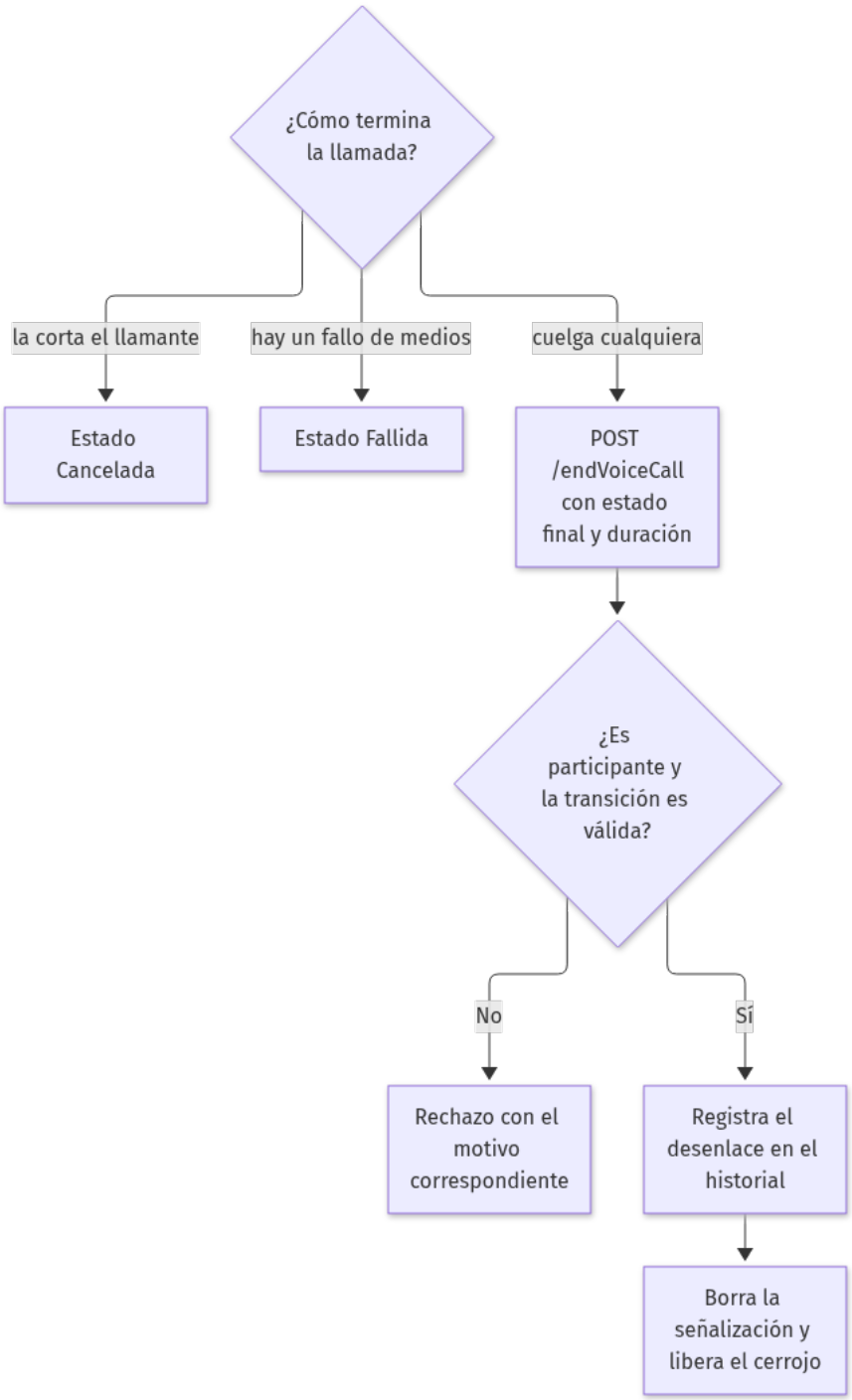


Figura 10

Diagrama de flujo del cierre de la llamada de voz y la actualización del historial.

5.10. Finalización y calificación

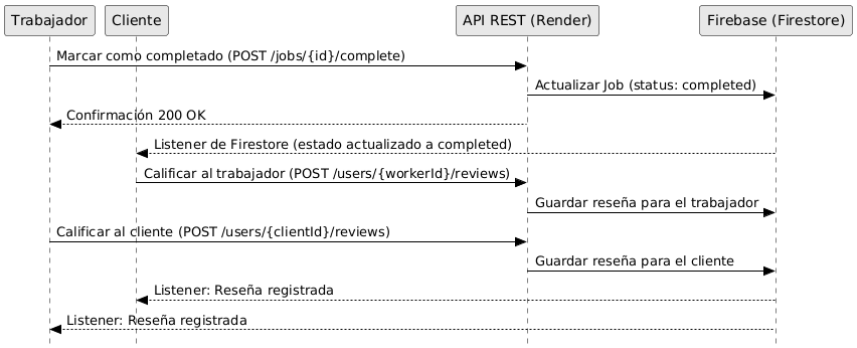


Figura 11

Diagrama de flujo del proceso de finalización del trabajo y calificación de las partes.

5.11. Pago simulado del trabajo

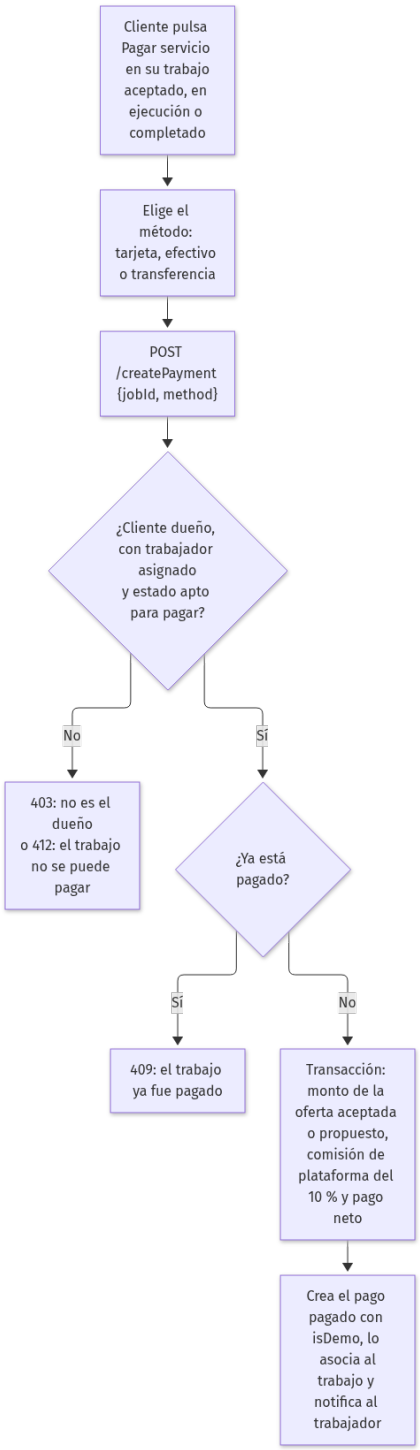


Figura 12

Diagrama de flujo del registro del pago simulado del trabajo y de las validaciones que lo acompañan.

5.12. Notificaciones

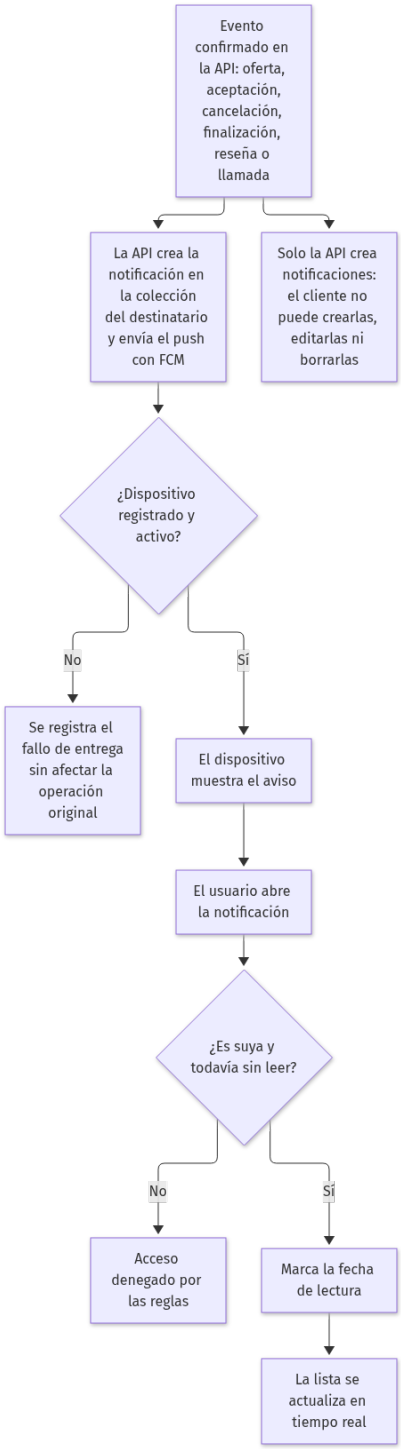


Figura 13

Diagrama de flujo de la creación, la entrega y la lectura de una notificación.

5.13. Administración del sistema

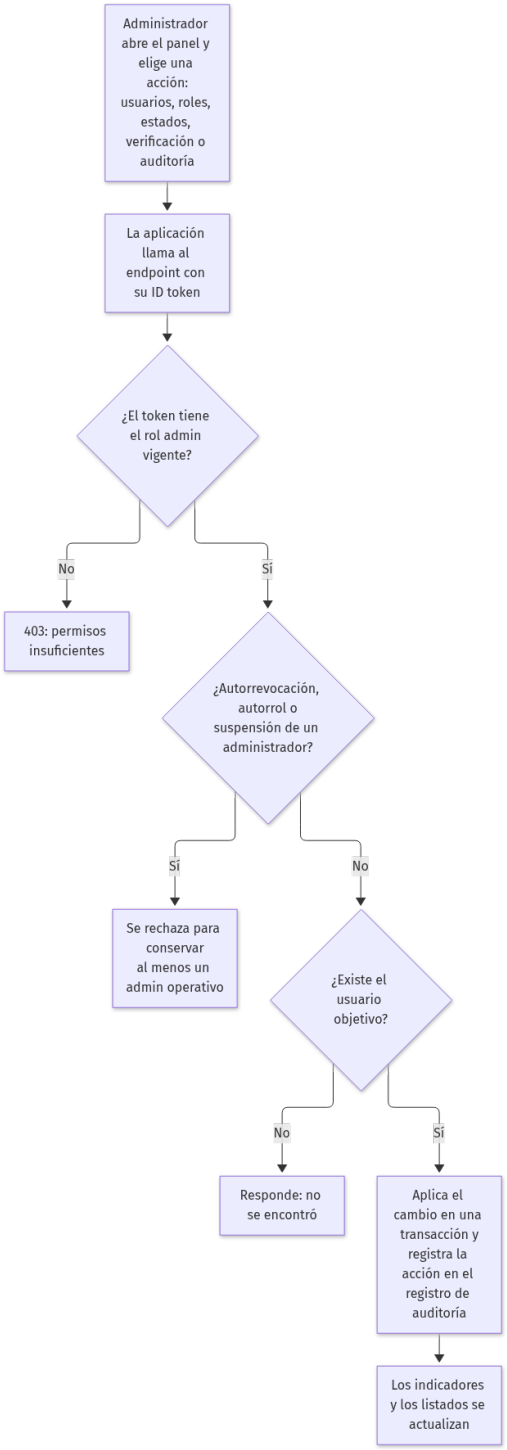


Figura 14

Diagrama de flujo de una operación administrativa y de las verificaciones de autorización que la preceden.

5.14. Flujo general del sistema

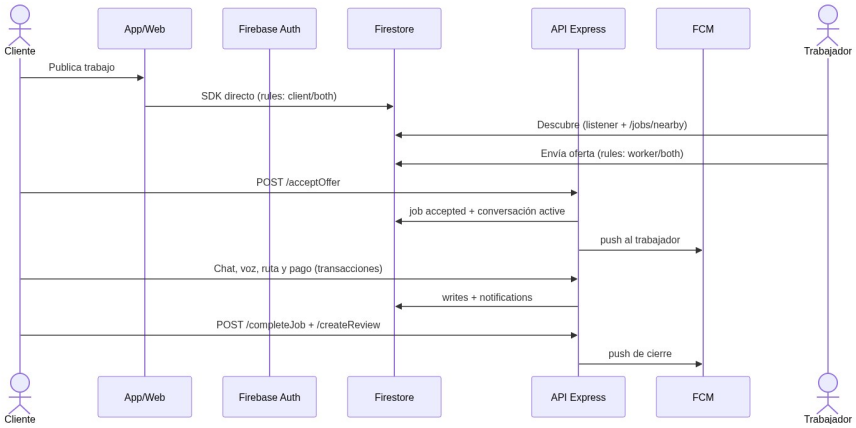


Figura 15

Diagrama de flujo general de la interacción entre cliente, trabajador, plataforma y servicios de soporte.

5.15. Ciclo de vida del trabajo

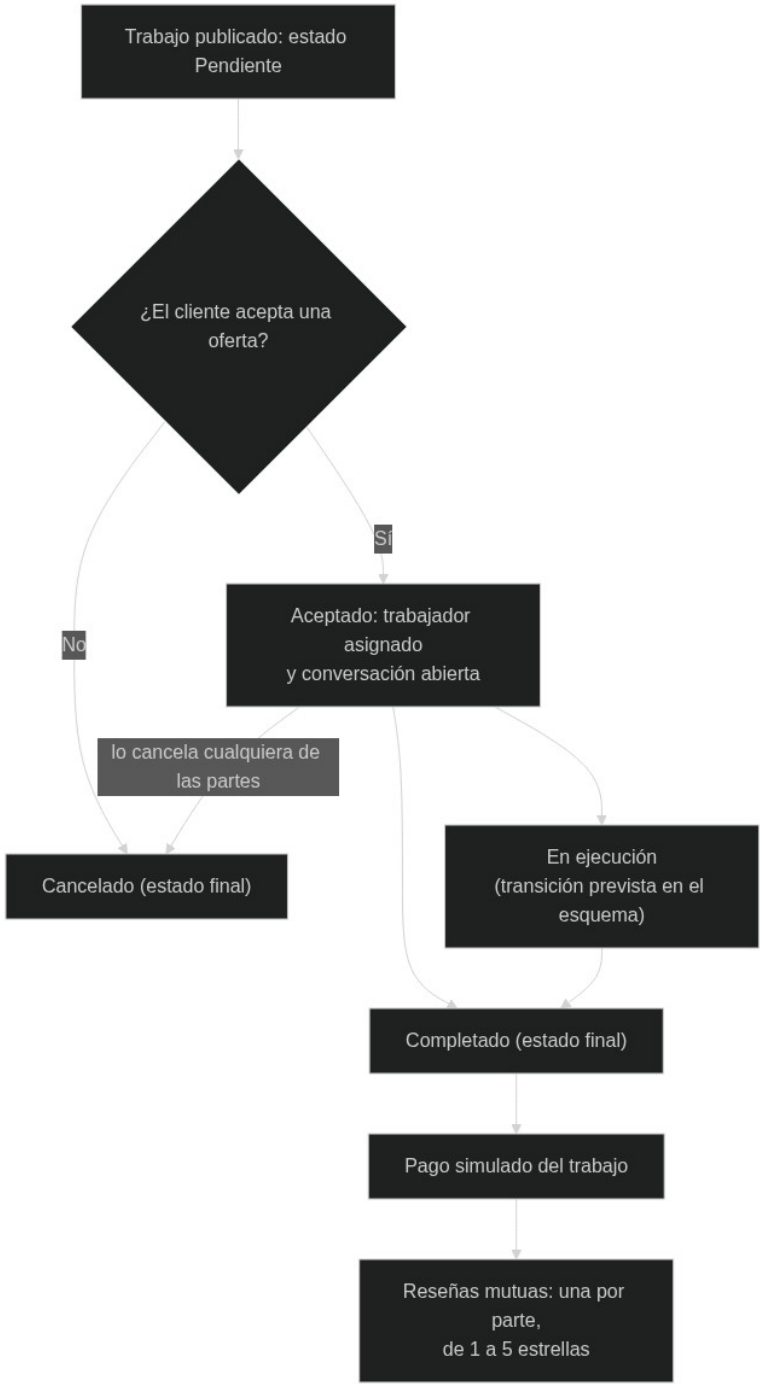


Figura 16

Diagrama del ciclo de vida de un trabajo: pendiente, aceptado, en ejecución, completado o cancelado, y los pasos posteriores de pago y calificación.

6. Casos de uso

6.1. Diagrama de casos de uso del sistema

El sistema se representa en cuatro diagramas, uno por actor: cliente, trabajador, ambos y administrador. Cada caso de uso se dibuja en una sola columna para que las líneas de asociación no se crucen sobre los óvalos, y los casos compartidos por más de un rol aparecen en el diagrama de cada uno de ellos.

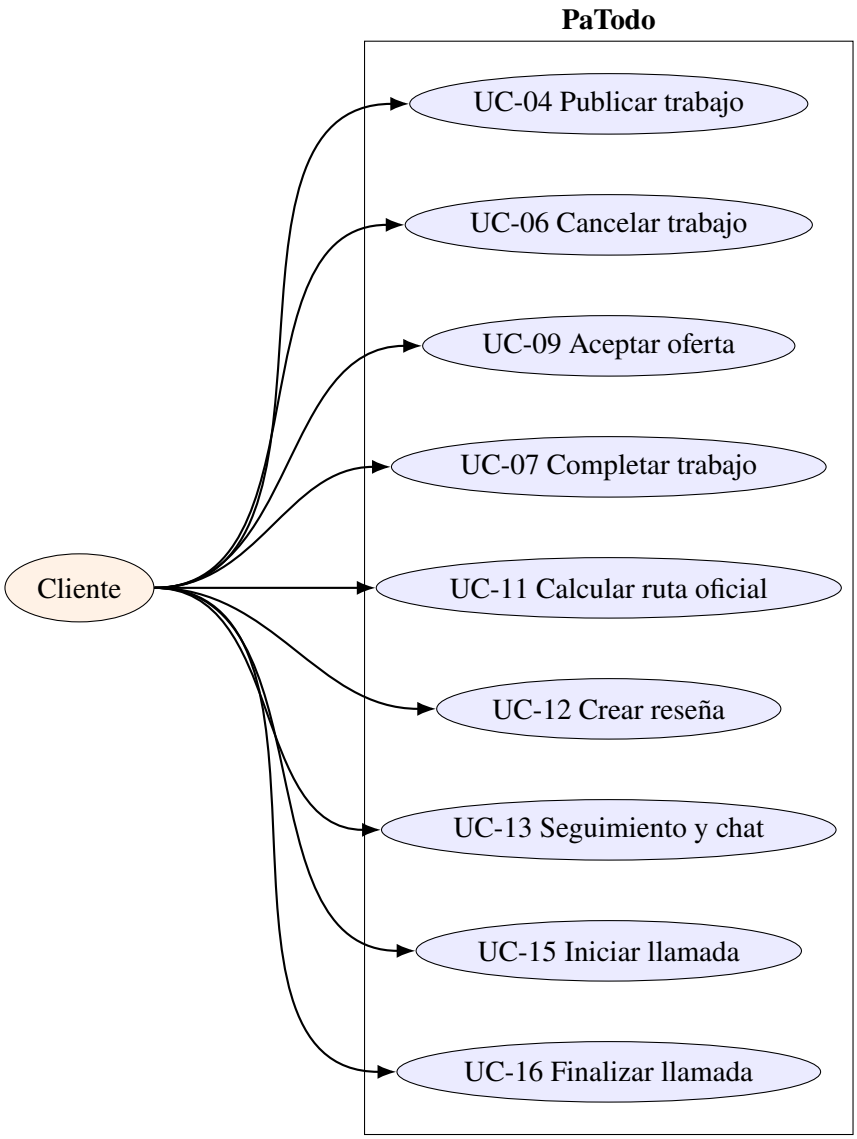


Figura 17

Diagrama de casos de uso del cliente: publica trabajos, gestiona las ofertas recibidas, da seguimiento, finaliza y califica.

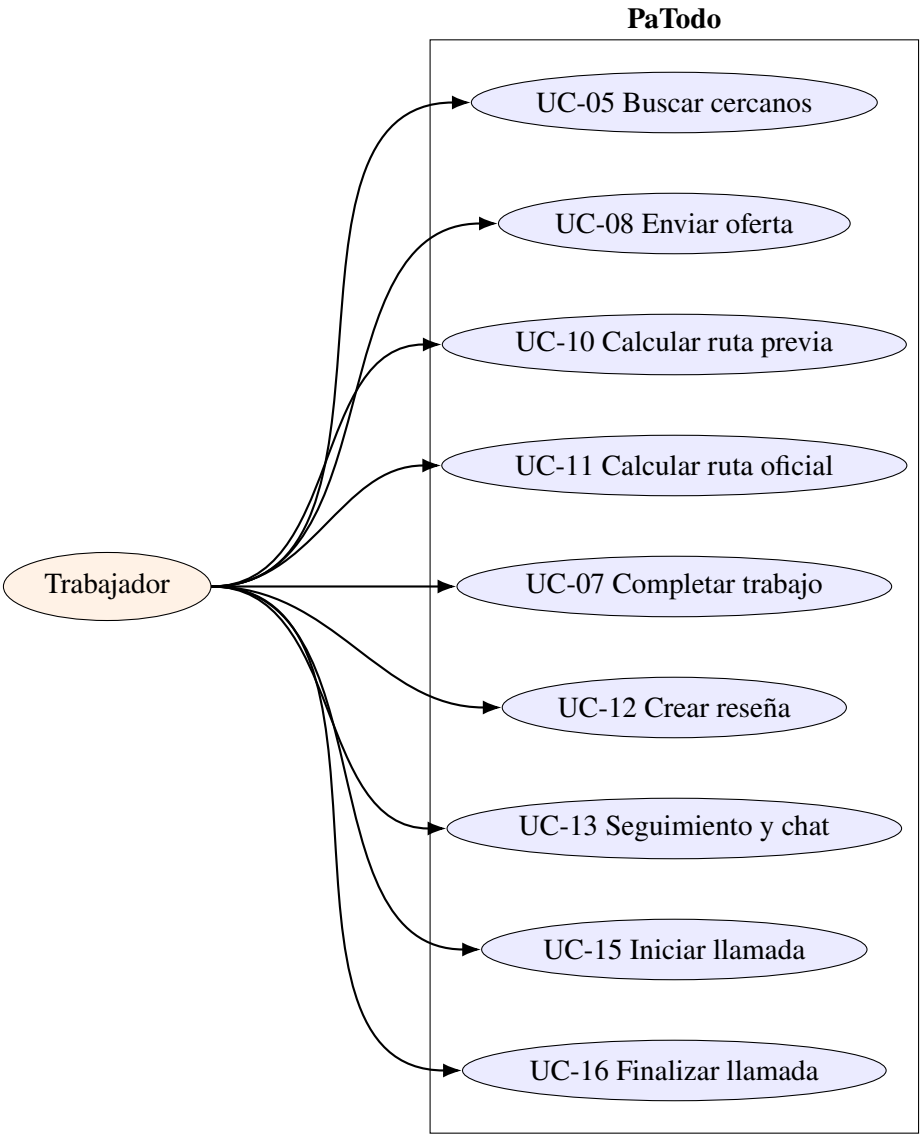


Figura 18

Diagrama de casos de uso del trabajador: busca trabajos cercanos, oferta, calcula la ruta, reporta avances y finaliza.

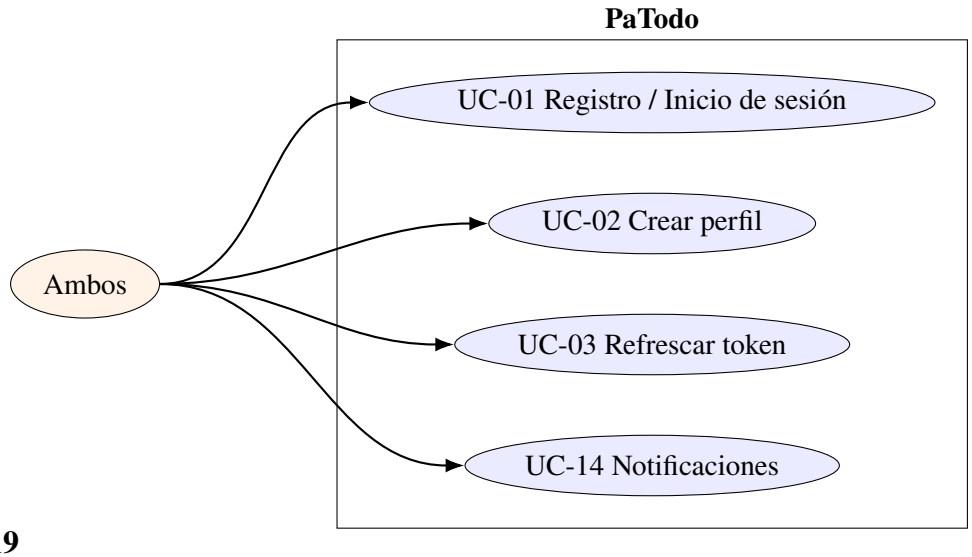


Figura 19

Diagrama de casos de uso de sesión y notificación, comunes a cualquier usuario autenticado, sea cual sea su rol.

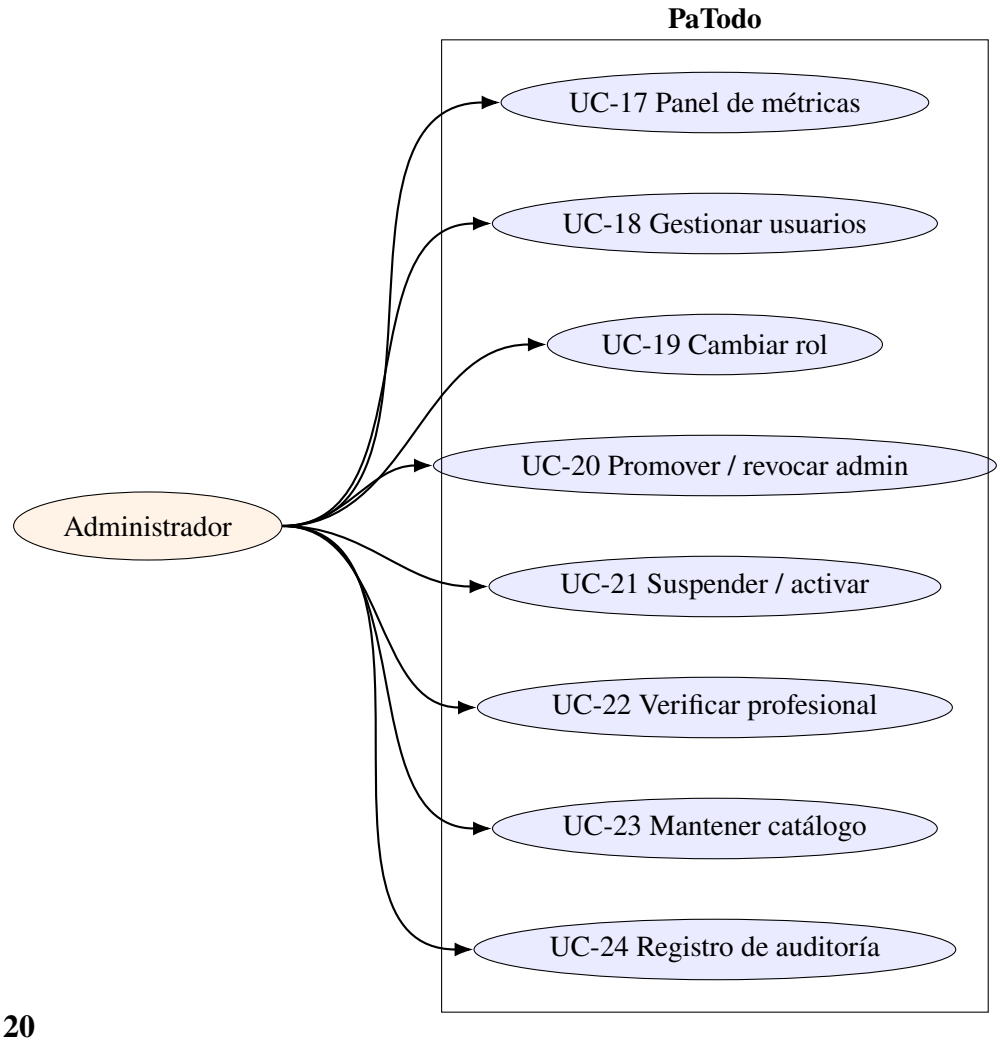


Figura 20

Diagrama de casos de uso de administración del sistema.

6.2. Casos de uso del módulo de autenticación

6.2.1. Caso de uso "UC-01: Registrarse / Iniciar sesión"

Descripción	El usuario crea una cuenta nueva o inicia sesión con su correo electrónico y contraseña, o bien mediante su cuenta de Google.
Actores primarios	Cliente, Trabajador.
Contexto	Pantalla de acceso del sistema.
Condiciones previas	El usuario no tiene una sesión iniciada.

Flujo básico	1. El usuario ingresa a la aplicación y selecciona la opción de registro o de inicio de sesión. 2. El usuario elige el método de autenticación: correo electrónico y contraseña, o Google. 3. Si elige Google, el sistema abre la ventana de consentimiento y recibe el resultado de la autenticación del proveedor. 4. Si elige correo y contraseña, el usuario ingresa su correo y su contraseña. 5. El sistema verifica las credenciales contra el servicio de autenticación. 6. Si las credenciales son válidas, el sistema genera un token de sesión y lo entrega a la aplicación. 7. Si el método es Google y la cuenta es nueva, el sistema solicita completar el perfil (teléfono y rol). 8. El sistema muestra la pantalla de bienvenida correspondiente al rol seleccionado.
Requerimientos especiales	Las contraseñas nunca se almacenan en el cliente ni se envían a la API. La sesión se mantiene mediante el token de identidad.
Salida	El usuario cuenta con sesión iniciada y token de identidad válido.
Excepciones	Credenciales inválidas: se muestra un mensaje de autenticación fallida sin revelar si el correo existe. Fallo del proveedor de Google: se muestra un mensaje de error y se ofrece reintentar.

6.2.2. Caso de uso "UC-02: Crear perfil de usuario"

Descripción	El usuario autenticado completa su perfil y selecciona el rol con el que desea operar.
Actores primarios	Cliente, Trabajador, Ambos.
Contexto	Módulo de perfil.
Condiciones previas	El usuario debe haber iniciado sesión.
Flujo básico	1. La aplicación solicita el nombre, el teléfono y el rol deseado. 2. El usuario ingresa sus datos y selecciona el rol: cliente, trabajador o ambos. 3. Opcionalmente, el trabajador registra sus habilidades y sus vehículos. 4. La aplicación envía la solicitud de creación de perfil al servicio REST. 5. El servicio verifica el token, comprueba que el identificador y el correo coincidan con los del token y comprueba que el perfil no exista previamente. 6. El servicio crea el documento del usuario con su rol y asigna el rol como declaración personalizada en el token de identidad. 7. La aplicación refresca el token de identidad y habilita las funciones del rol elegido.
Requerimientos especiales	El identificador y el correo de la solicitud deben coincidir con los del token; de lo contrario se rechaza la operación. El teléfono es obligatorio.
Salida	El documento de usuario creado con su rol, y las acciones del rol habilitadas.

Excepciones	Perfil ya existente: el servicio responde `409` y la aplicación dirige al usuario a editar su perfil.

6.2.3. Caso de uso "UC-03: Refrescar token de identidad"

Descripción	El usuario renueva su token de identidad para que las reglas de seguridad y la API reconozcan su rol y sus privilegios actuales.
Actores primarios	Cliente, Trabajador, Ambos, Administrador.
Contexto	Inicio de sesión, creación o edición del perfil, cambio de rol, o después de un rechazo de escritura por permisos.
Condiciones previas	El usuario debe tener una sesión iniciada.
Flujo básico	1. La aplicación detecta que necesita un token vigente: al abrir la app, tras crear o editar el perfil, tras cambiar de rol, o cuando una escritura es rechazada con un error de permisos. 2. La aplicación solicita el refresco forzado del token al proveedor de autenticación. 3. El proveedor emite un token nuevo que incluye las declaraciones personalizadas actualizadas (rol y privilegio de administrador). 4. La aplicación utiliza el token nuevo en sus operaciones con Firestore y con la API REST.
Requerimientos especiales	Las reglas de Firestore leen el rol del token, por lo que un token emitido antes de crear el perfil o de cambiar de rol no refleja los permisos vigentes y las escrituras se rechazan aunque el usuario esté autenticado. El refresco forzado es obligatorio después de esas operaciones.

Salida	Token de identidad vigente con el rol y los privilegios actualizados.
Excepciones	Sin conexión: se reintenta. Sesión revocada o expirada que no puede renovarse: la aplicación cierra la sesión y solicita autenticarse de nuevo.

6.3. Casos de uso del módulo de trabajos

6.3.1. Caso de uso "UC-04: Publicar trabajo"

Descripción	El cliente publica una solicitud de servicio para recibir ofertas de trabajadores cercanos.
Actores primarios	Cliente.
Contexto	Módulo de trabajos.
Condiciones previas	El usuario debe tener rol cliente o ambos, su cuenta no debe estar suspendida y su perfil debe estar completo.

Flujo básico	1. El usuario accede a la opción de publicar un trabajo. 2. El sistema carga el catálogo de categorías y habilidades, de lectura pública. 3. El usuario describe el trabajo, selecciona la categoría y las habilidades requeridas. 4. El usuario indica el lugar de la recogida; la aplicación obtiene las coordenadas del dispositivo o permite introducir la dirección. 5. Opcionalmente, el usuario indica un punto de entrega y la fecha en que requiere el servicio. 6. El usuario indica el precio que está dispuesto a pagar. 7. La aplicación escribe el trabajo directamente en la base de datos, asignándolo al usuario y en estado pendiente. 8. El sistema muestra la confirmación de publicación y el trabajo queda disponible para los trabajadores cercanos.
Requerimientos especiales	El trabajo debe incluir los datos mínimos obligatorios: cliente, descripción, ubicación, precio y estado. La ubicación debe incluir el valor de geocodificación para que el trabajo aparezca en las búsquedas por proximidad.
Salida	Trabajo publicado en estado pendiente y visible en la búsqueda de trabajos cercanos.
Excepciones	El usuario no cuenta con permisos para publicar: se muestra un mensaje de permisos insuficientes. Los campos obligatorios incompletos se señalan en el formulario.

6.3.2. Caso de uso “UC-05: Buscar trabajos cercanos”

Descripción	El trabajador consulta los trabajos pendientes próximos a su ubicación, filtrados por distancia y, opcionalmente, por categoría.
Actores primarios	Trabajador.
Contexto	Módulo de trabajos cercanos.
Condiciones previas	El usuario debe tener rol trabajador o ambos, su cuenta no debe estar suspendida y debe contar con ubicación disponible.
Flujo básico	1. El usuario abre la pantalla de trabajos cercanos. 2. La aplicación obtiene las coordenadas del dispositivo y solicita la búsqueda con el radio configurado, por defecto 10 kilómetros. 3. Opcionalmente, el usuario ajusta el radio de búsqueda o selecciona una categoría. 4. El servicio calcula los límites geográficos del área solicitada y consulta los trabajos pendientes dentro de esos límites. 5. El servicio descarta los trabajos cuya distancia real supera el radio y los ordena de menor a mayor distancia. 6. La aplicación muestra la lista de trabajos con la distancia en línea recta, la categoría, el precio propuesto y la fecha requerida.
Requerimientos especiales	El radio máximo admitido es de 50 kilómetros y el número máximo de resultados es de 50. La distancia mostrada no representa la ruta por carretera.
Salida	Listado de trabajos cercanos con su distancia, ordenado de menor a mayor.

Excepciones	Un cliente que intenta la búsqueda recibe un mensaje de permisos insuficientes. Coordenadas o radio con formato inválido: se rechaza la solicitud indicando los parámetros incorrectos. Sin trabajos en el área: se muestra un mensaje informativo.

6.3.3. Caso de uso "UC-06: Cancelar trabajo"

Descripción	El cliente cancela un trabajo que ya no necesita, siempre que no haya sido completado.
Actores primarios	Cliente.
Contexto	Módulo de trabajos.
Condiciones previas	El usuario debe ser el cliente dueño del trabajo y el trabajo debe encontrarse en estado pendiente o aceptado.
Flujo básico	1. El usuario localiza el trabajo en su lista y selecciona la opción de cancelar. 2. Opcionalmente, el usuario indica el motivo de la cancelación. 3. El sistema solicita la confirmación. 4. El usuario confirma y la aplicación envía la solicitud al servicio REST. 5. El servicio verifica que el usuario sea el dueño del trabajo y que su estado permita la cancelación. 6. El servicio registra la cancelación, libera los recursos asociados (seguimiento y conversación) y notifica a las partes involucradas.
Requerimientos especiales	Un trabajo completado no puede cancelarse. La cancelación se ejecuta únicamente a través del servicio REST.

Salida	Trabajo en estado cancelado y notificación a la contraparte.
Excepciones	El usuario no es el dueño del trabajo o el estado no permite la cancelación: se rechaza la operación con el motivo correspondiente.

6.3.4. Caso de uso "UC-07: Completar trabajo"

Descripción	Una vez prestado el servicio, el cliente o el trabajador asignado marcan el trabajo como completado.
Actores primarios	Cliente, Trabajador.
Contexto	Módulo de trabajos.
Condiciones previas	El trabajo debe encontrarse en estado aceptado y el usuario debe ser el cliente o el trabajador asignado.
Flujo básico	1. El usuario abre el detalle del trabajo y selecciona la opción de completar. 2. El sistema solicita la confirmación. 3. El usuario confirma y la aplicación envía la solicitud al servicio REST. 4. El servicio verifica la participación del usuario y el estado del trabajo. 5. El servicio registra la finalización con la fecha, libera el seguimiento de ubicación y genera las notificaciones para ambas partes. 6. Ambas partes pueden proceder a calificar el servicio.
Requerimientos especiales	La finalización solo es válida para trabajos aceptados. Un trabajo cancelado no puede completarse.

Salida	Trabajo en estado completado, con la fecha de finalización registrada.
Excepciones	Usuario no participante o estado inválido: se rechaza la operación.

6.4. Casos de uso del módulo de ofertas

6.4.1. Caso de uso "UC-08: Enviar oferta"

Descripción	El trabajador envía una oferta para un trabajo cercano, con un precio, un tiempo estimado y un mensaje para el cliente.
Actores primarios	Trabajador.
Contexto	Módulo de ofertas.
Condiciones previas	El usuario debe tener rol trabajador o ambos, su cuenta no debe estar suspendida y el trabajo debe estar pendiente y pertenecer a otro cliente.
Flujo básico	1. El trabajador selecciona un trabajo del listado de cercanos y revisa su detalle, distancia y precio propuesto. 2. Opcionalmente, consulta la vista previa de la ruta para valorar el trayecto. 3. El trabajador selecciona la opción de ofertar. 4. El trabajador ingresa el precio que propone, el tiempo estimado de ejecución y un mensaje para el cliente. 5. La aplicación escribe la oferta directamente en la base de datos, en estado pendiente. 6. El cliente recibe una notificación de la nueva oferta y puede revisarla junto con las demás.

Requerimientos especiales	La oferta debe incluir trabajo, trabajador, precio, tiempo estimado y estado. No se puede ofertar por un trabajo propio ni por un trabajo que ya no esté pendiente. No es posible modificar el trabajo ni el trabajador de la oferta.
Salida	Oferta registrada en estado pendiente y notificación al cliente.
Excepciones	El trabajo pertenece al propio trabajador, no está pendiente o el usuario no tiene permisos: se rechaza la oferta con el motivo correspondiente.

6.4.2. Caso de uso "UC-09: Aceptar oferta"

Descripción	El cliente selecciona la mejor oferta, asigna al trabajador y abre la conversación de coordinación.
Actores primarios	Cliente.
Contexto	Detalle del trabajo, sección de ofertas recibidas.
Condiciones previas	El usuario debe ser el cliente dueño del trabajo, el trabajo debe estar pendiente y debe existir al menos una oferta.

Flujo básico	1. El cliente abre el detalle del trabajo y revisa las ofertas recibidas (precio, tiempo estimado, mensaje, perfil y calificación del trabajador). 2. El cliente selecciona la oferta que desea aceptar. 3. El sistema solicita la confirmación. 4. El cliente confirma y la aplicación envía la solicitud al servicio REST. 5. El servicio ejecuta una transacción que marca la oferta seleccionada como aceptada, el resto de ofertas como rechazadas y el trabajo como aceptado, registrando al trabajador asignado. 6. El servicio crea la conversación privada entre ambas partes en estado activa. 7. El servicio notifica la asignación al trabajador y notifica al cliente.
Requerimientos especiales	La operación es atómica: si la transacción falla, ninguno de los cambios se aplica, evitando trabajos aceptados sin trabajador asignado.
Salida	Trabajo aceptado con trabajador asignado, conversación creada y notificaciones enviadas.
Excepciones	El trabajo ya fue aceptado por otro usuario: la operación se rechaza por conflicto. El usuario no es el dueño del trabajo: se rechaza por permisos insuficientes.

6.5. Casos de uso del módulo de rutas

6.5.1. Caso de uso "UC-10: Calcular ruta (vista previa)"

Descripción	El trabajador no asignado consulta la distancia y el trayecto desde su ubicación hasta un trabajo pendiente, para decidir si le interesa.
Actores primarios	Trabajador.
Contexto	Detalle de un trabajo pendiente.
Condiciones previas	El usuario debe tener rol trabajador o ambos, el trabajo debe estar pendiente y el trabajador debe tener una ubicación válida.
Flujo básico	1. El trabajador solicita calcular la ruta hacia el trabajo. 2. La aplicación envía la solicitud al servicio REST. 3. El servicio verifica el rol y el estado del trabajo, y comprueba la disponibilidad de la ubicación del trabajador. 4. El servicio consulta al motor de cálculo de rutas la geometría, la distancia y la duración del trayecto. 5. La aplicación muestra la ruta trazada sobre el mapa, con la distancia y el tiempo estimado.
Requerimientos especiales	En este modo **no se persiste ningún dato** del trabajo ni del trabajador.
Salida	Vista previa de la ruta mostrada en pantalla.
Excepciones	El trabajador no tiene ubicación disponible: se rechaza la operación indicando que debe actualizar su ubicación. El trabajo ya está aceptado: se rechaza por estado inválido. El motor de rutas no responde: se informa que el servicio no está disponible temporalmente.

6.5.2. Caso de uso "UC-11: Calcular ruta oficial"

Descripción	Con el trabajo ya aceptado, se calcula y almacena la ruta definitiva que sigue el trabajador, dividida en el trayecto hacia la recogida y, si existe, hacia el punto de entrega.
Actores primarios	Cliente, Trabajador.
Contexto	Detalle de un trabajo aceptado.
Condiciones previas	El trabajo debe estar aceptado y el usuario debe ser el cliente dueño o el trabajador asignado.
Flujo básico	1. El usuario solicita la ruta del trabajo aceptado. 2. El servicio verifica la participación del usuario y el estado del trabajo. 3. El servicio obtiene la ubicación actual del trabajador asignado y las coordenadas del trabajo. 4. El servicio consulta al motor de rutas la geometría, la distancia y la duración; si existe destino, divide el trayecto en dos tramos. 5. El servicio almacena el resultado en el campo de ruta del trabajo. 6. La aplicación muestra la ruta y la distancia estimada de llegada.
Requerimientos especiales	La ruta almacenada incluye geometría, distancia, duración, tramos, origen del dato y fecha de cálculo.
Salida	Ruta oficial almacenada en el trabajo y disponible para el cliente.
Excepciones	El trabajo no está aceptado o el usuario no participa: se rechaza la operación. Falta de ubicación del trabajador: se rechaza indicando que debe actualizar su ubicación.

6.6. Casos de uso del módulo de finalización

6.6.1. Caso de uso "UC-12: Crear reseña"

Descripción	Tras completar el trabajo, el cliente y el trabajador califican mutuamente el servicio prestado.
Actores primarios	Cliente, Trabajador.
Contexto	Detalle de un trabajo completado.
Condiciones previas	El trabajo debe estar completado y el usuario debe haber participado en él sin haber publicado ya una reseña.
Flujo básico	1. El usuario abre el detalle de un trabajo completado y selecciona la opción de calificar. 2. El usuario asigna una puntuación de una a cinco estrellas. 3. Opcionalmente, el usuario escribe un comentario. 4. La aplicación envía la calificación al servicio REST. 5. El servicio verifica la participación, el estado del trabajo y que no exista ya una reseña del mismo autor para ese trabajo. 6. El servicio registra la reseña, recalcula la calificación promedio y el número de reseñas del perfil del evaluado. 7. El servicio notifica la nueva reseña a la contraparte.
Requerimientos especiales	La puntuación va de 1 a 5. Las reseñas son inmutables: una vez registradas no pueden modificarse ni eliminarse.
Salida	Reseña registrada, calificación promedio del perfil actualizada y notificación a la contraparte.
Excepciones	El usuario no participó en el trabajo, el trabajo no está completado o ya lo ha calificado previamente: se rechaza la operación indicando el motivo.

6.7. Casos de uso del módulo de seguimiento y chat

6.7.1. Caso de uso "UC-13: Seguimiento en tiempo real y chat"

Descripción	Durante la ejecución del trabajo, el trabajador reporta su ubicación y el cliente la sigue en tiempo real; además, ambas partes se comunican por mensajes dentro de la conversación del trabajo.
Actores primarios	Cliente, Trabajador.
Contexto	Detalle del trabajo en ejecución.
Condiciones previas	El trabajo debe estar aceptado y existir una conversación asociada.
Flujo básico	1. El trabajador inicia el trayecto y activa el reporte de ubicación. 2. La aplicación escribe la posición actual del trabajador en el documento de seguimiento del trabajo. 3. El cliente, con el detalle del trabajo abierto, observa la posición actualizada automáticamente. 4. El trabajador puede enviar mensajes desde el chat del trabajo. 5. La aplicación escribe el mensaje en la conversación y actualiza el resumen del último mensaje. 6. El cliente recibe la actualización del chat en tiempo real y puede responder.
Requerimientos especiales	El documento de seguimiento es único por trabajo (posición actual, sin historial) y se elimina al completar o cancelar. Solo el cliente dueño y el trabajador asignado pueden leer y escribir en el seguimiento. La ubicación precisa no se publica en el perfil general del usuario.

Salida	Posición del trabajador visible en tiempo real y conversación actualizada con los mensajes enviados.
Excepciones	Un tercero que intenta leer el seguimiento o la conversación: se rechaza por permisos insuficientes. Un mensaje con contenido vacío o superior al límite permitido se rechaza en el cliente.

6.8. Casos de uso del módulo de notificaciones

6.8.1. Caso de uso "UC-14: Recibir y consultar notificaciones"

Descripción	El sistema avisa al usuario, mediante una notificación push y una entrada en su bandeja, sobre los eventos de los trabajos, ofertas, mensajes, reseñas, pagos y llamadas de voz en los que participa.
Actores primarios	Cliente, Trabajador.
Actores secundarios	API REST (emisora), Firebase Cloud Messaging (entrega push).
Contexto	Bandeja de notificaciones; eventos generados por la API durante las operaciones transaccionales.
Condiciones previas	El usuario debe tener sesión iniciada. Para recibir push es necesario que su perfil tenga al menos un token FCM registrado; sin tokens, el aviso queda únicamente en la bandeja.

Flujo básico	1. Ocurre un evento para el usuario: nueva oferta en un trabajo publicado, oferta aceptada o rechazada, trabajo iniciado, completado o cancelado, mensaje nuevo, reseña recibida, pago registrado o llamada de voz entrante. 2. La API guarda el aviso en la colección `notifications` y envía el push por FCM a los tokens del usuario; la llamada entrante usa prioridad alta para que el teléfono la atienda de inmediato. 3. La aplicación muestra el aviso según su tipo y, en una llamada entrante, presenta la pantalla de llamada con la orientación correspondiente. 4. El usuario abre la notificación; la aplicación navega al recurso relacionado (trabajo, oferta, conversación o llamada) y marca el aviso como leído. 5. El usuario consulta el historial en su bandeja de notificaciones.
Requerimientos especiales	Tipos soportados: `new_offer`, `offer_accepted`, `offer_rejected`, `new_message`, `job_started`, `job_completed`, `job_cancelled`, `new_review`, `payment_received` y `voice_call_incoming`. La notificación es un efecto secundario de la transacción: si la entrega falla, el estado del trabajo o de la oferta ya quedó guardado y solo se registra el error. El push de llamada entrante se envía con prioridad alta y un TTL corto.
Salida	Aviso visible en la bandeja y push entregado; el evento relacionado queda registrado en Firestore.

Excepciones	El usuario no tiene tokens FCM: solo se guarda el aviso en la bandeja. Tokens inválidos o dados de baja: la API los elimina del perfil automáticamente. El dispositivo apagado o sin red: FCM reintenta la entrega según su política.

6.9. Casos de uso del módulo de comunicación por voz

6.9.1. Caso de uso "UC-15: Iniciar llamada de voz"

Descripción	El cliente o el trabajador asignado inician una llamada de voz directa con la contraparte del trabajo en curso.
Actores primarios	Cliente, Trabajador.
Contexto	Detalle del trabajo en ejecución.
Condiciones previas	El trabajo debe estar aceptado o en curso, el usuario debe participar en él y no debe existir una llamada activa para ese trabajo.

Flujo básico	1. El usuario selecciona la opción de llamar a la contraparte. 2. La aplicación solicita al servicio REST la creación de la sesión de voz. 3. El servicio verifica el estado del trabajo, la participación del usuario, que ninguno esté suspendido y que no exista ya una llamada activa. 4. El servicio registra la llamada, toma el cerrojo del trabajo y devuelve el identificador de la sesión, la contraparte y los servidores de conexión con credenciales de tiempo limitado. 5. La contraparte recibe una notificación de llamada entrante. 6. La contraparte acepta y ambas partes intercambian los mensajes de señalización en la base de datos. 7. Se establece la conexión de audio directa entre los dispositivos.
Requerimientos especiales	Solo puede existir una llamada activa por trabajo; el cerrojo del servidor garantiza esta restricción incluso ante intentos simultáneos. Las credenciales de conexión son de corta duración y el secreto del servidor de relevo reside únicamente en el servidor. El audio viaja cifrado y **nunca** se almacena. No se permiten llamadas a números telefónicos.
Salida	Llamada establecida entre las partes, con sesión registrada en el historial.
Excepciones	Ya existe una llamada activa para el trabajo: se responde conflicto y se indica que hay una llamada en curso. El usuario no participa del trabajo o está suspendido: se rechaza por permisos insuficientes. La contraparte no responde dentro del tiempo de espera: la llamada se cierra como perdida y se libera el cerrojo.

6.9.2. Caso de uso "UC-16: Finalizar llamada de voz"

Descripción	Cualquiera de las dos partes cuelga la llamada y el sistema registra el desenlace.
Actores primarios	Cliente, Trabajador.
Contexto	Llamada de voz en curso.
Condiciones previas	Debe existir una llamada activa y el usuario debe ser parte de ella.
Flujo básico	1. El usuario cuelga la llamada. 2. La aplicación notifica el cierre al otro participante mediante la señalización. 3. La aplicación solicita al servicio REST el cierre de la sesión, indicando el estado final y la duración. 4. El servicio verifica la participación y que la transición de estado sea válida. 5. El servicio registra el estado final y la duración, elimina los mensajes de señalización y libera el cerrojo del trabajo. 6. El historial de llamadas queda disponible para el usuario.
Requerimientos especiales	Solo se admiten transiciones de estado válidas. Una llamada en curso nunca se cierra automáticamente por el barrido de llamadas perdidas.
Salida	Llamada cerrada y registrada en el historial; señalización y cerrojo eliminados.
Excepciones	El usuario no forma parte de la llamada: se rechaza la operación. La transición solicitada no es válida: se rechaza indicando el estado actual.

6.10. Casos de uso del módulo de administración

6.10.1. Caso de uso "UC-17: Consultar panel de métricas"

Descripción	El administrador consulta el tablero principal con los indicadores de operación de la plataforma.
Actores primarios	Administrador.
Contexto	Módulo de administración.
Condiciones previas	El usuario debe contar con el rol administrador vigente en su token.
Flujo básico	1. El administrador ingresa al panel de administración. 2. La aplicación solicita las métricas al servicio REST. 3. El servicio verifica el rol administrador. 4. El servicio agrega el número de usuarios por rol, el número de trabajos por estado, el total de ofertas, la calificación promedio y la actividad de la última semana. 5. La aplicación presenta los indicadores en tarjetas y tablas.
Requerimientos especiales	Los indicadores de ingresos y de usuarios activos se devuelven en cero mientras el sistema no incorpore pagos ni registro de sesión; el tablero no debe interpretarlos como datos reales.
Salida	Tablero de métricas de la plataforma.
Excepciones	El usuario no es administrador: se rechaza la operación por permisos insuficientes.

6.10.2. Caso de uso "UC-18/UC-19: Gestionar usuarios y roles"

Descripción	El administrador consulta el listado de usuarios, aplica filtros de búsqueda y modifica el rol operativo de cada cuenta.

Actores primarios	Administrador.
Contexto	Módulo de administración, sección de usuarios.
Condiciones previas	El usuario debe contar con el rol administrador vigente en su token.
Flujo básico	1. El administrador abre la sección de usuarios. 2. Opcionalmente, aplica filtros por rol, por estado o por texto de búsqueda, y selecciona la página. 3. La aplicación solicita el listado paginado al servicio REST. 4. El servicio verifica el rol, aplica los filtros y devuelve los resultados. 5. El administrador selecciona un usuario y elige un nuevo rol: cliente, trabajador o ambos. 6. El servicio verifica el rol administrador, actualiza el rol del usuario y registra la acción en el registro de auditoría. 7. El usuario afectado debe refrescar su token para que el nuevo rol surta efecto.
Requerimientos especiales	El cambio de rol operativo no puede usarse para promover a administrador; la promoción requiere la operación específica. Los cambios no pueden aplicarse sobre el propio administrador.
Salida	Listado filtrado de usuarios y rol actualizado.
Excepciones	El usuario objetivo no existe: se responde que no se encontró. El llamante no es administrador: se rechaza por permisos insuficientes.

6.10.3. Caso de uso "UC-20: Promover / revocar administrador"

Descripción	El administrador otorga el rol administrador a una cuenta existente o lo revoca, degradándola a cliente.
Actores primarios	Administrador.
Contexto	Módulo de administración, sección de usuarios.
Condiciones previas	El usuario debe contar con el rol administrador vigente en su token. Debe existir al menos un administrador inicial, creado mediante la utilidad de arranque del servidor.
Flujo básico	1. El administrador selecciona un usuario del listado y elige promover o revocar el rol administrador. 2. El sistema solicita la confirmación, indicando el efecto de la acción. 3. El administrador confirma y la aplicación envía la solicitud al servicio REST. 4. El servicio verifica el rol administrador, aplica el cambio y registra la acción en el registro de auditoría. 5. El usuario afectado debe refrescar su token para que el rol surta efecto.
Requerimientos especiales	Un administrador **no puede revocarse a sí mismo**, para garantizar la existencia de al menos un administrador operativo.
Salida	Rol administrador actualizado y acción registrada en el registro de auditoría.
Excepciones	Intento de auto-revocación: se rechaza indicando que no es posible. Usuario inexistente: se responde que no se encontró.

6.10.4. Caso de uso "UC-21: Suspender / activar cuenta"

Descripción	El administrador suspende temporalmente una cuenta para impedir su operación, o la reactiva.
Actores primarios	Administrador.
Contexto	Módulo de administración, sección de usuarios.
Condiciones previas	El usuario debe contar con el rol administrador vigente en su token.
Flujo básico	1. El administrador selecciona un usuario y elige la opción de suspender o activar. 2. El sistema solicita la confirmación y explica el efecto de la acción. 3. El administrador confirma y la aplicación envía la solicitud al servicio REST. 4. El servicio verifica el rol, actualiza el estado de la cuenta y registra la acción en el registro de auditoría. 5. La cuenta suspendida conserva la lectura, pero toda escritura suya es rechazada; al reactivarla recupera su operación normal.
Requerimientos especiales	No se puede suspender a una cuenta con rol administrador. Las cuentas creadas antes de existir el campo de estado se consideran activas.
Salida	Estado de la cuenta actualizado y acción registrada.
Excepciones	Se intenta suspender a un administrador: se rechaza la operación por permisos insuficientes. Usuario inexistente: se responde que no se encontró.

6.10.5. Caso de uso "UC-22: Verificar profesional"

Descripción	El administrador revisa a los trabajadores pendientes de verificación y marca o retira la condición de profesional verificado.
Actores primarios	Administrador.
Contexto	Módulo de administración, sección de verificación.
Condiciones previas	El usuario debe contar con el rol administrador vigente en su token.
Flujo básico	1. El administrador abre la sección de verificación, que lista los trabajadores no verificados. 2. Selecciona un trabajador y revisa la información de su perfil. 3. El administrador decide aprobar o rechazar la verificación. 4. El sistema solicita la confirmación y luego envía la solicitud al servicio REST. 5. El servicio verifica el rol, actualiza la marca de verificación y registra la acción en el registro de auditoría.
Requerimientos especiales	La marca de verificación solo puede modificarla el administrador.
Salida	Trabajador marcado como verificado (o como no verificado) y acción registrada.
Excepciones	El usuario objetivo no existe: se responde que no se encontró. El llamante no es administrador: se rechaza por permisos insuficientes.

6.10.6. Caso de uso "UC-23: Mantener catálogo de categorías y habilidades"

Descripción	El administrador crea, edita, activa, desactiva y elimina categorías y habilidades del catálogo de servicios.

Actores primarios	Administrador.
Contexto	Módulo de administración, sección de catálogo.
Condiciones previas	El usuario debe contar con el rol administrador vigente en su token.
Flujo básico	1. El administrador abre la sección de catálogo y elige categorías o habilidades. 2. Selecciona la opción de crear un registro nuevo o de editar uno existente. 3. El formulario exige el código, el nombre y el estado activo; en el caso de las habilidades, las categorías asociadas. 4. El administrador guarda los cambios. 5. El sistema valida los campos y persiste el registro directamente en la base de datos, autorizado por las reglas de seguridad para el rol administrador. 6. El sistema confirma la operación y actualiza el listado y los formularios de publicación.
Requerimientos especiales	El identificador de cada registro corresponde a su código, por lo que debe ser único y estable. La lectura del catálogo es pública, pero su escritura está reservada al administrador.
Salida	Catálogo actualizado y disponible para los formularios de publicación y los filtros de búsqueda.
Excepciones	Un usuario distinto del administrador intenta modificar el catálogo: se rechaza por permisos insuficientes.

6.10.7. Caso de uso "UC-24: Consultar registro de auditoría"

Descripción	El administrador consulta el registro de las acciones administrativas realizadas en la plataforma.
Actores primarios	Administrador.
Contexto	Módulo de administración, sección de auditoría.
Condiciones previas	El usuario debe contar con el rol administrador vigente en su token.
Flujo básico	1. El administrador abre la sección de auditoría. 2. Opcionalmente, indica el número de registros a mostrar. 3. La aplicación solicita el registro al servicio REST. 4. El servicio verifica el rol y devuelve las acciones registradas: quién las ejecutó, qué acción realizó, sobre qué entidad y su descripción. 5. La aplicación presenta el listado en orden cronológico.
Requerimientos especiales	El registro solo puede ser escrito por el servicio y solo puede ser leído por administradores. El número de registros solicitado tiene un límite máximo.
Salida	Listado de acciones administrativas registradas.
Excepciones	El usuario no es administrador: se rechaza por permisos insuficientes.

7. Diagrama de secuencia

7.1. Secuencia de publicación de un trabajo

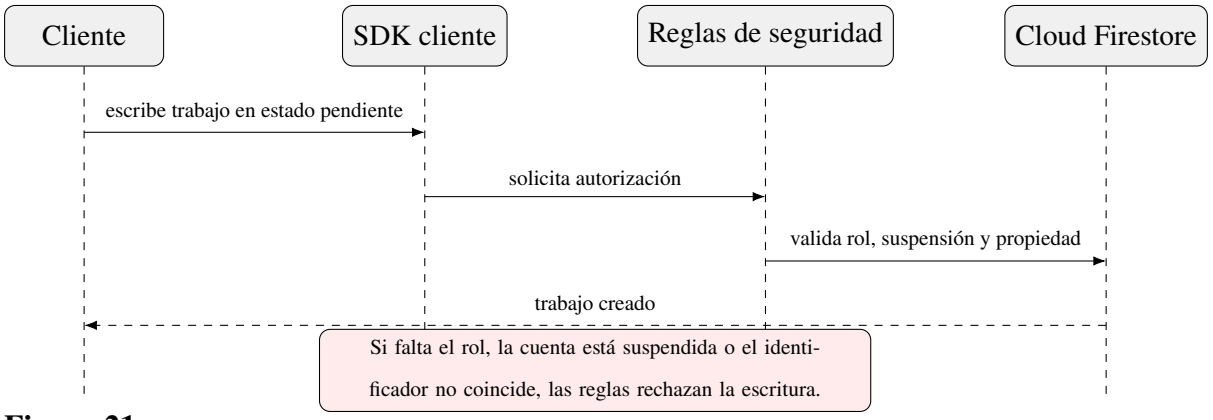


Figura 21

Secuencia de publicación de un trabajo con validación en las reglas de seguridad.

7.2. Secuencia de aceptación de oferta

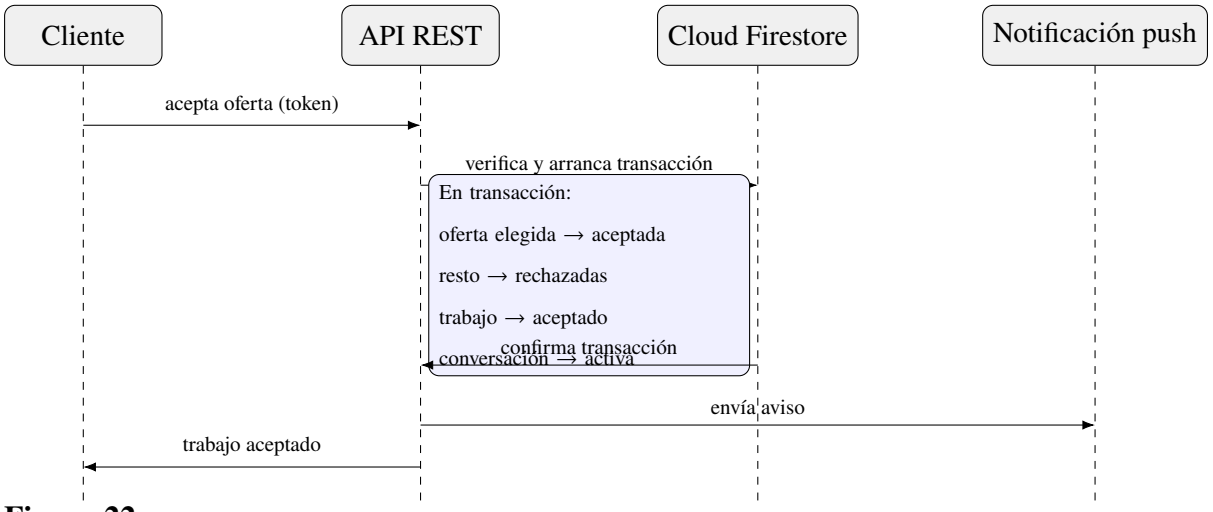


Figura 22

Secuencia de aceptación de oferta y asignación del trabajador.

7.3. Secuencia de finalización y calificación

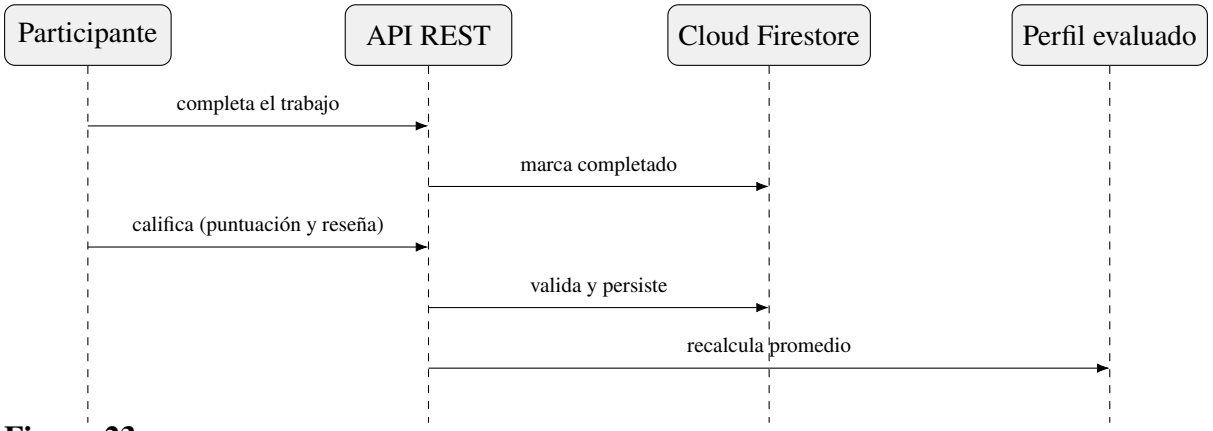


Figura 23

Secuencia de finalización del trabajo y creación de la reseña.

7.4. Secuencia de la búsqueda de trabajos cercanos

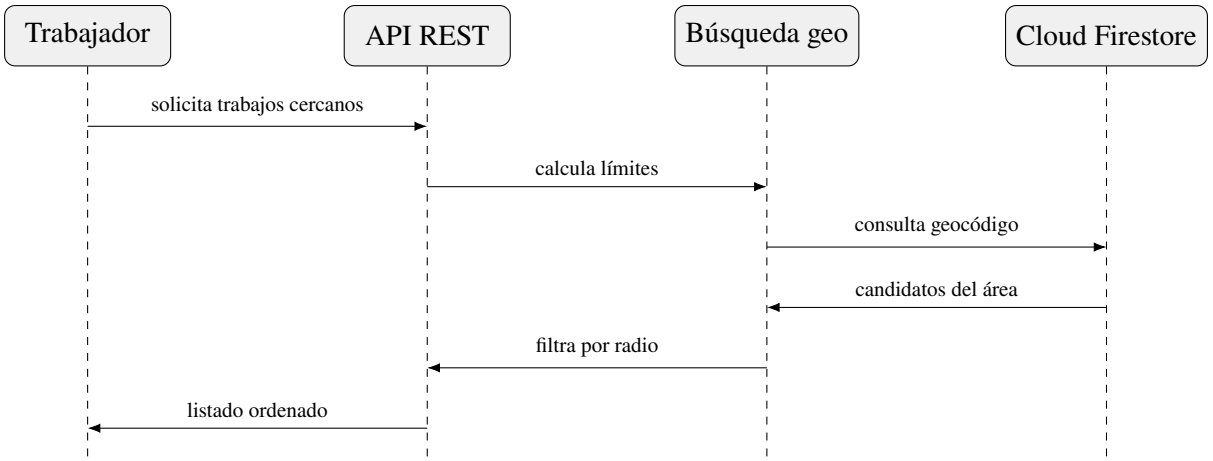


Figura 24

Secuencia de la búsqueda de trabajos cercanos por proximidad geográfica.

8. Diagramas de estado

8.1. Estados del trabajo

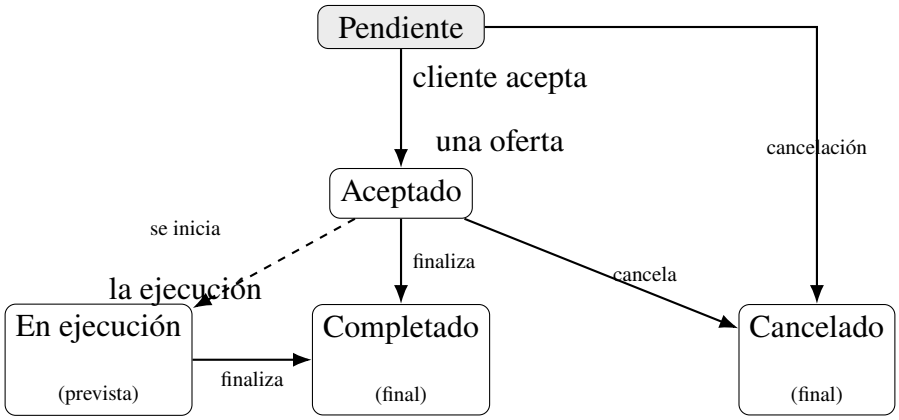


Figura 25

Diagrama de estados de un trabajo. Los estados finales son completado y cancelado. La transición hacia en ejecución está prevista en el esquema de datos, pero la API no la asigna automáticamente: un trabajo aceptado puede completarse directamente.

8.2. Estados de la oferta

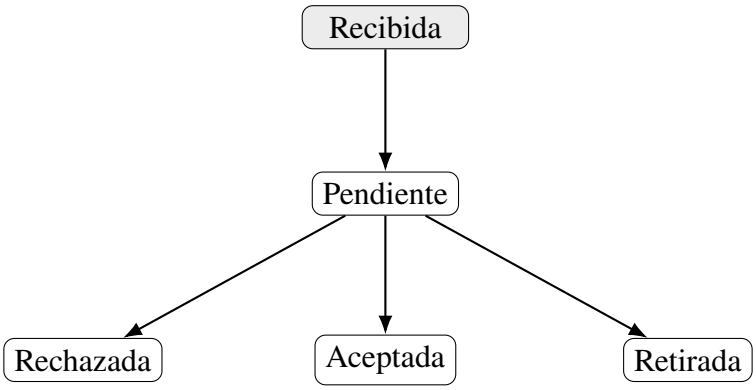


Figura 26

Diagrama de estados de una oferta. Los estados rechazada, retirada y aceptada son finales.

8.3. Estados de la llamada de voz

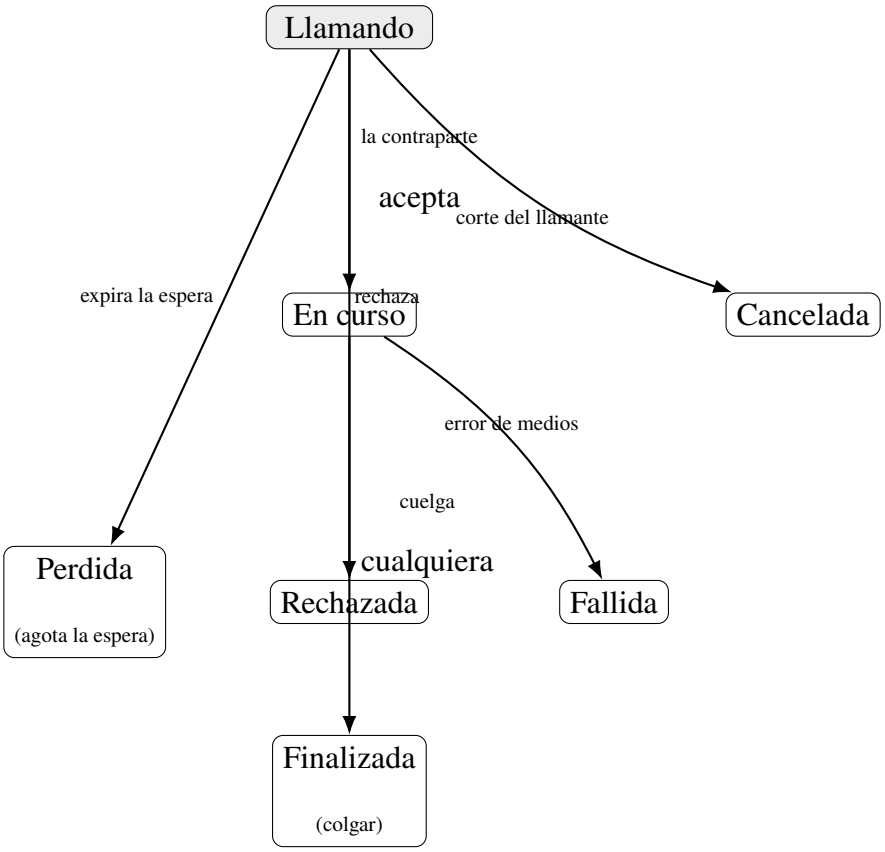


Figura 27

Diagrama de estados de una llamada de voz. El cerrojo por trabajo garantiza una única llamada activa.

9. Diagrama ER

El modelo de datos se organiza en colecciones de documentos, con referencias por identificador entre entidades. La siguiente figura representa las entidades principales y sus relaciones.

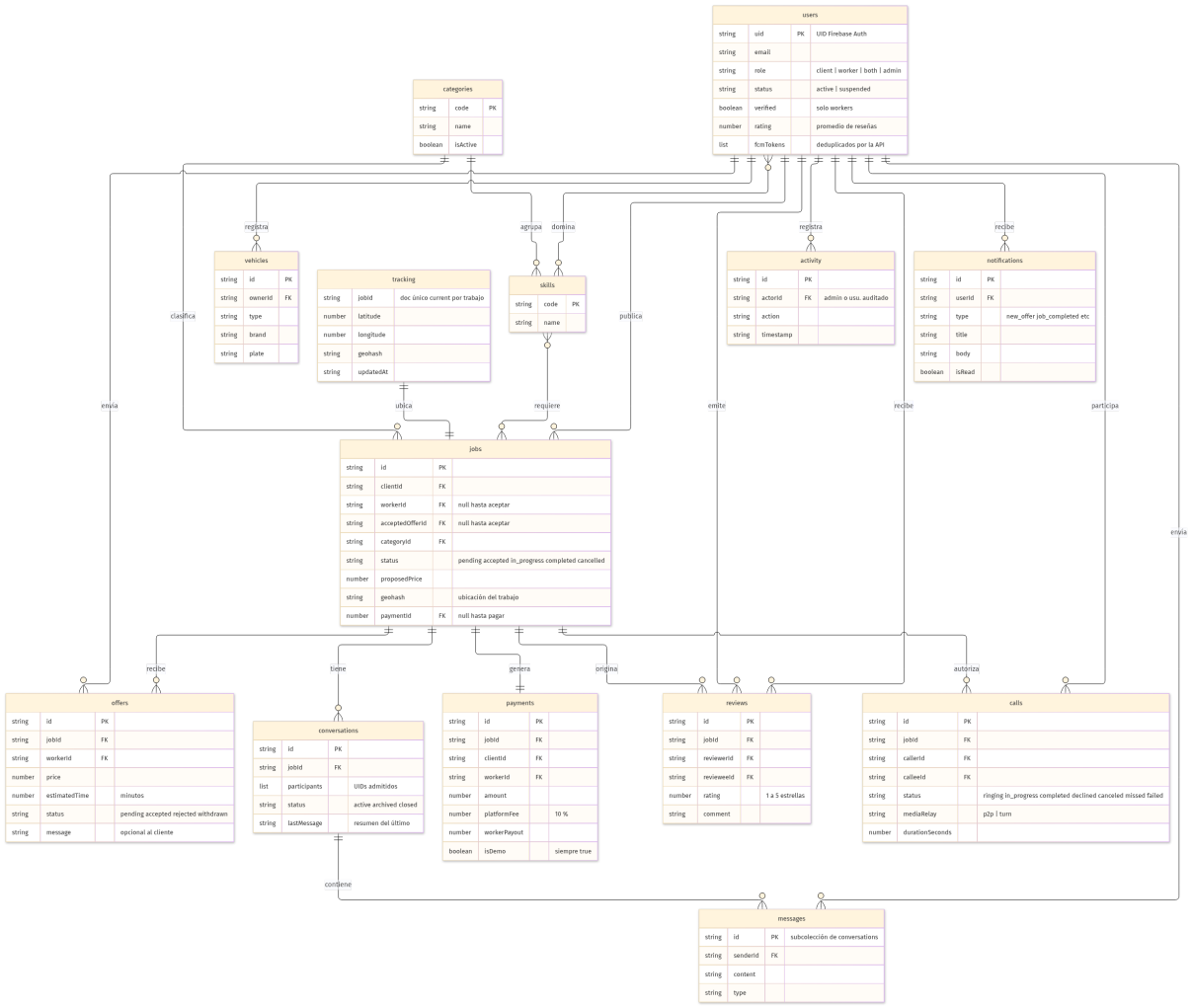


Figura 28

Diagrama entidad-relación de PaTodo (modelo de documentos en Cloud Firestore). Los nombres de las entidades corresponden a las colecciones y los campos a los atributos principales; las relaciones indican referencias por identificador que las reglas de seguridad y la API mantienen consistentes (Firestore no impone claves foráneas). `messages` *es subcolección de* `conversations` *y* `tracking` *(documento único* `current`*) de* `jobs`*.*

10. Requerimientos no funcionales

10.1. Arquitectura de tres capas

El desarrollo de la aplicación debe seguir una arquitectura de tres capas. En la capa de **base de datos** se utiliza un DBMS de documentos en la nube (Cloud Firestore), con consultas geográficas y reglas de seguridad declarativas. En la capa de **backend** se implementan

servicios REST en Node.js con el framework Express y el lenguaje TypeScript, responsable de las operaciones transaccionales y de las notificaciones; los servicios de autenticación y de mensajería push se consumen como servicios gestionados (Firebase Authentication y Firebase Cloud Messaging). En la capa de **frontend** se desarrollan dos aplicaciones cliente: una aplicación web de una sola página con React y TypeScript, y una aplicación móvil con Flutter para Android y iOS, ambas con comunicación con la base de datos a través de los SDK de cliente y con la capa de backend mediante el consumo de los servicios REST.

10.2. Autenticación y autorización

Acceso al sistema. La autenticación se realiza mediante el servicio de autenticación de Firebase, que admite correo electrónico con contraseña y inicio de sesión con Google. Tras la autenticación, el sistema emite un *token de identidad* que la capa de backend verifica en cada petición mediante la cabecera de autorización. Los roles de usuario son *cliente*, *trabajador*, *ambos* y *administrador*; el rol se almacena en el perfil del usuario y se replica como declaración personalizada en el token, de modo que las reglas de seguridad puedan aplicarlo sin consultar la base de datos.

Reglas de autorización por rol y por recurso.

- Solo los roles cliente y ambos pueden publicar trabajos; solo los trabajadores y ambos pueden enviar ofertas y registrar vehículos.
- Cada usuario solo puede modificar su propio perfil, sus propios vehículos y sus propias ofertas.
- Solo el cliente dueño del trabajo puede aceptarlo, editarlo, cancelarlo o eliminarlo mientras esté pendiente.
- Solo los participantes de un trabajo pueden leer su conversación y escribir mensajes en ella, y solo el cliente dueño y el trabajador asignado pueden leer y escribir el seguimiento de ubicación.
- Las colecciones de servidor (reseñas, notificaciones y conversaciones) solo pueden ser creadas por la API.

- Los catálogos de categorías y habilidades son de lectura pública y de escritura exclusiva del administrador.
- El registro de auditoría es de escritura exclusiva de la API y de lectura exclusiva del administrador.
- Toda escritura de una cuenta suspendida es rechazada; la lectura se conserva.

Control de acceso a datos. Solo los usuarios con los permisos necesarios pueden acceder a la información de los demás. Un usuario no puede consultar conversaciones, notificaciones ni seguimientos en los que no participa.

10.3. Seguridad de datos

Protección de credenciales. Las contraseñas son gestionadas por el servicio de autenticación con su propio mecanismo de derivación; el sistema no almacena ni transmite contraseñas. Las claves de servicio del servidor se inyectan únicamente mediante variables de entorno y nunca se versionan en el repositorio. La clave de configuración del cliente no es un secreto y puede ir en el paquete de la aplicación.

Protección de las comunicaciones. Todas las comunicaciones entre el cliente, la API y los servicios de Firebase se realizan mediante HTTPS con cifrado en tránsito. Las conexiones de voz entre las partes se establecen mediante cifrado en el transporte y el audio nunca se almacena en el servidor.

Capa de API.. La API aplica validación explícita del cuerpo de cada petición y devuelve un formato de error uniforme con el estado HTTP correspondiente. Aplica además un límite de peticiones por dirección de origen para mitigar ataques de denegación de servicio.

Integridad. Los datos de cada proceso están protegidos contra modificaciones no autorizadas: las transiciones de estado, la asignación del trabajador, la ruta calculada, las reseñas, las notificaciones y el registro de auditoría solo pueden ser alterados por la API, y cada operación transaccional es atómica.

Disponibilidad. El sistema debe estar disponible de forma continua. Los servicios de Firebase y el alojamiento de la aplicación web y de la API ofrecen niveles de servicio

comerciales que garantizan su disponibilidad.

10.4. Almacenamiento

Capacidad. El almacenamiento de datos es el de un servicio gestionado con escalado automático, por lo que no requiere dimensionamiento manual de disco. El crecimiento de la operación (usuarios, trabajos, ofertas, mensajes, reseñas y notificaciones) se atiende sin intervención del equipo.

Rendimiento. Las consultas simples sobre documentos y las consultas geográficas deben responder en tiempos compatibles con una experiencia de uso interactiva; las lecturas de listados se paginan para evitar respuestas de tamaño excesivo. El cálculo de rutas debe resolverse en un tiempo que no bloquee la interfaz. Las operaciones que requieren despertar el servicio tras un periodo de inactividad deben mostrar al usuario una espera informativa y no cancelable.

Disponibilidad e integridad de los archivos. Las fotografías de perfil y de trabajos se almacenan en el servicio de almacenamiento con reglas de acceso coherentes con los permisos del sistema; los catálogos de categorías y habilidades son legibles públicamente, mientras que el resto de los archivos solo es accesible a su propietario.

Límites de tamaño. Todo documento almacenado tiene un límite de tamaño para evitar el abuso del espacio y la denegación de servicio; los documentos que lo excedan son rechazados.

10.5. Integración

Integración con servicios de terceros. El sistema se integra con:

- El servicio de autenticación y la base de datos de Firebase, mediante los SDK de cliente para las operaciones propias y mediante la API propia para las operaciones transaccionales.
- El servicio de notificaciones push de Firebase, para el envío de avisos a los dispositivos.
- Un motor de cálculo de rutas, consultado por la API REST para obtener la geometría, la distancia y la duración del trayecto.

- La plataforma de publicación en la nube, para el despliegue de la aplicación web y de las reglas e índices de la base de datos.

Contratos de integración. Toda operación de la API se documenta formalmente en la especificación OpenAPI 3, que constituye la fuente única de verdad de los contratos. Los modelos de datos se especifican en esquemas JSON que validan la estructura de cada colección. No se admiten operaciones fuera de la especificación.

Formato de datos. Los datos se intercambian en formato JSON y los documentos de base de datos respetan el esquema de cada colección; los nombres de colección y campos siguen una convención única (colecciones en plural, campos en notación *camelCase*).

11. Planificación de ciclos de desarrollo

Ciclo	**Qué abarca**	**Fechas**
Uno	Especificación de los modelos de datos y de los contratos de la interfaz (esquemas JSON y especificación OpenAPI), estructura del repositorio y configuración de los servicios en la nube (autenticación, base de datos, reglas e índices).	Semanas 1–3
Dos	Construcción de la API REST transaccional: autenticación de peticiones, manejo uniforme de errores, publicación de trabajos, búsqueda geográfica, envío y aceptación de ofertas, cancelación, finalización, reseñas y cálculo de rutas.	Semanas 4–5

Ciclo	Qué abarca	Fechas
Tres	Funcionalidades de tiempo real: sincronización por listeners, mensajería entre las partes, notificaciones push y seguimiento de ubicación; además, endurecimiento de las reglas de seguridad con autorización por rol y por propiedad del recurso.	Semanas 6–7
Cuatro	Interfaz web (React) e interfaz móvil (Flutter): autenticación, panel de trabajos, detalle y publicación, ofertas, mensajería, perfil, notificaciones y mapas.	Semanas 8–11
Cinco	Módulo de administración: métricas, gestión de usuarios y roles, suspensión y activación, verificación de profesionales, catálogo y registro de auditoría.	Semanas 12–13
Seis	Comunicación de voz entre las partes (sesiones de llamada, señalización persistida, cerrojo por trabajo y conexión directa), y su integración en las interfaces.	Semanas 14–15
Siete	Pruebas de extremo a extremo de la interfaz, validación de los contratos y de las reglas de seguridad, y correcciones derivadas de la verificación.	Semanas 16–17
Ocho	Despliegue en producción de la interfaz web, de la API y de las reglas e índices, y publicación de la aplicación móvil.	Semana 18

11.1. Estado de avance

Componente	**Estado**
Especificación de modelos de datos e interfaz	Completado
Configuración de los servicios en la nube y reglas de seguridad	Completado
API REST transaccional	Completado
Funcionalidades de tiempo real, mensajería y notificaciones	Completado
Interfaz web	Completado y publicado
Interfaz móvil	En desarrollo
Módulo de administración	Completado
Comunicación de voz	Backend completado y probado; interfaz móvil en desarrollo

12. Firmas de aceptación

Las partes firman en señal de conformidad con el contenido de este documento de especificaciones, requerimientos y criterios de aceptación del sistema PaTodo.

Nombre y puesto	Firma y sello	Fecha

Documento generado a partir de la especificación del proyecto (`spec/openapi.yaml` y `spec/schemas/`) y de la documentación técnica en `docs/`.

Anexo A. Catálogo de la interfaz REST

Método	Ruta	Descripción
GET	`/`	Health check del servicio.
POST	`/createUser`	Crea el perfil del usuario autenticado y asigna su rol.
POST	`/acceptOffer`	Acepta una oferta: asigna al trabajador y abre la conversación.
POST	`/cancelJob`	Cancela un trabajo pendiente o aceptado.
POST	`/completeJob`	Marca un trabajo aceptado como completado.
POST	`/createReview`	Registra la calificación de un trabajo completado.
POST	`/computeRoute`	Calcula la ruta del trabajo: vista previa u oficial.
POST	`/createPayment`	Registra el pago del servicio.
POST	`/createVoiceSession`	Crea una sesión de llamada de voz y toma el cerrojo.
POST	`/endVoiceCall`	Cierra la llamada y libera el cerrojo.
GET	`/jobs/nearby`	Busca trabajos pendientes cercanos por proximidad.
GET	`/admin/stats`	Métricas de operación del tablero.
GET	`/admin/users`	Listado paginado y filtrado de usuarios.
PATCH	`/admin/users/{uid}/role`	Cambia el rol operativo de un usuario.

Método	Ruta	Descripción
POST	`/admin/makeAdmin`	Promueve una cuenta a administrador.
POST	`/admin/removeAdmin`	Revoca el rol administrador.
POST	`/admin/suspendUser`	Suspende una cuenta.
POST	`/admin/activateUser`	Reactiva una cuenta.
POST	`/admin/verifyWorker`	Verifica o deja de verificar a un profesional.
GET	`/admin/jobs`	Listado de trabajos con filtro por estado.
GET	`/admin/activityLog`	Registro de acciones administrativas.

Anexo B. Estados y sus transiciones

Entidad	Estados y transiciones permitidas
Trabajo	*Pendiente* → *Aceptado* (al aceptar una oferta); *Pendiente* → *Cancelado*; *Aceptado* → *En ejecución* (transición prevista en el esquema, hoy no se asigna automáticamente); *Aceptado* → *Completado*; *En ejecución* → *Completado*; *Aceptado* → *Cancelado*. Los estados *Completado* y *Cancelado* son finales.
Pago	*Pagado* al registrarse el pago simulado del trabajo: único estado previsto, sin transiciones posteriores.
Oferta	*Pendiente* → *Aceptada* (al aceptar el cliente esa oferta); *Pendiente* → *Rechazada* (al aceptar el cliente otra oferta); *Pendiente* → *Retirada* (por el propio trabajador). No se admiten otras transiciones.
Conversación	*Activa* al crearse al aceptar la oferta.

Entidad	**Estados y transiciones permitidas**
Llamada de voz	*Llamando* → *En curso* (aceptación); *Llamando* → *Perdida* (agotamiento de la espera); *Llamando* → *Rechazada*; *Llamando* → *Cancelada* (corte del llamante); *En curso* → *Finalizada* (cuelgue de cualquiera de las partes); *En curso* → *Fallida* (error de medios).
Cuenta de usuario	*Activa* ↔ *Suspendida* (por el administrador).